

# Multi-objective optimization for distributed flexible job shop scheduling problem with job priority

Ao He, Xiahui Liu<sup>\*</sup>, Guiliang Gong<sup>\*</sup>, Zhipeng Yuan, Hongbo Huang, Yang Zhou, Jie Li

College of Mechanical and Intelligent Manufacturing, Central South University of Forestry and Technology, Changsha 410004, China

## ARTICLE INFO

### Keywords:

Distributed flexible job shop scheduling  
Job priority  
Memetic algorithm  
Local search

## ABSTRACT

For the distributed flexible job shop scheduling problem (DFJSP), the existing researches have predominantly focused on operation sequence, machine selection and factory assignment, and assuming that the jobs have no priority. However, in real-world manufacturing systems, production scheduling with job priority is very common and is of concern to production managers. The paper presents a DFJSP with job priority (DFJSPJP) for the first time, aiming at minimizing the makespan, total energy consumption and the weighted average time of jobs with priority. A new memetic algorithm (NMA) is designed to solve the proposed DFJSPJP. In the proposed NMA, a well-designed chromosome encoding method (CEM) is constructed to obtain a high-quality initial population. An effective local search approach (LSO) is proposed to improve the NMA's convergence speed and fully exploit its solution space. Computational experiments conducted confirm the effectiveness of the CEM and LSO, and show that the NMA is able to easily obtain better solutions for about 90 % of the tested 60 challenging problem instances compared to other three well-known algorithms, demonstrating its superior performance on both solution quality and computational efficiency. This research will provide a theoretical basis for considering job priority issues in distributed production environments and assist manufacturers in conducting accurate production scheduling, thereby reducing resource waste and time loss caused by unreasonable production plans.

## 1. Introduction

In recent years, with the rise of economic globalization and the increasing diverse demands of customers, manufacturing systems are shifting from traditional highly centralized environments to more flexible distributed settings to allocate production resources sensibly and enhance production efficiency. Distributed flexible job shop scheduling (DFJSP) [1–5] has emerged as a prominent research focus because of its practical relevance and significance for managers to make production decisions in manufacturing systems. DFJSP involves the strategic arrangement of jobs, factories, workshops, and machines to derive viable solutions [6–9], encompassing four key sub-problems: (1) allocating each job to a factory from a set of optional factories; (2) assigning each job to a workshop selected from the optional workshops within the corresponding factory; (3) allotting each job to a machine chosen from the optional machines within the corresponding workshop of factory; and (4) sequencing operations while adhering to constraints across factories, machines, and operations concurrently [10,11]. Undoubtedly, DFJSP poses a more challenging NP-hard conundrum compared to the

traditional flexible job shop scheduling problem (FJSP) involving a single factory only [12–14].

Over the past few decades, DFJSP has attracted wide attention [15–18] from researchers due to its practical significance for decision-making by managers in manufacturing systems. Different researchers have extended this problem in various ways. So far, to the best of our knowledge, all of the studies on DFJSP focused on operation sequence, machine selection and factory assignment, and assuming that the jobs have no priority. However, in real-world manufacturing systems, production scheduling that incorporates job priorities is a prevalent phenomenon and a key concern for production managers.

To thoroughly elucidate the practical application context of the distributed flexible job shop scheduling problem (DFJSP) with job priorities, this paper provides a detailed explanation through Fig. 1. Among them, Fig. 1(a) illustrates a typical distributed production workshop layout for industrial robots. Fig. 1(b) presents the main structure of an industrial robot, in which the base is the highest-priority component. As the foundational support structure of industrial robots, the base serves as the assembly reference for all other components. Its manufacturing

<sup>\*</sup> Corresponding authors.

E-mail addresses: [liuxiahui@hnu.edu.cn](mailto:liuxiahui@hnu.edu.cn) (X. Liu), [guiliang\\_gong@163.com](mailto:guiliang_gong@163.com) (G. Gong), [t20192440@csuft.edu.cn](mailto:t20192440@csuft.edu.cn) (Z. Yuan), [hongbohuang@hnu.edu.cn](mailto:hongbohuang@hnu.edu.cn) (H. Huang).

<https://doi.org/10.1016/j.swevo.2025.102075>

Received 19 January 2025; Received in revised form 16 June 2025; Accepted 6 July 2025

Available online 18 July 2025

2210-6502/© 2025 Elsevier B.V. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

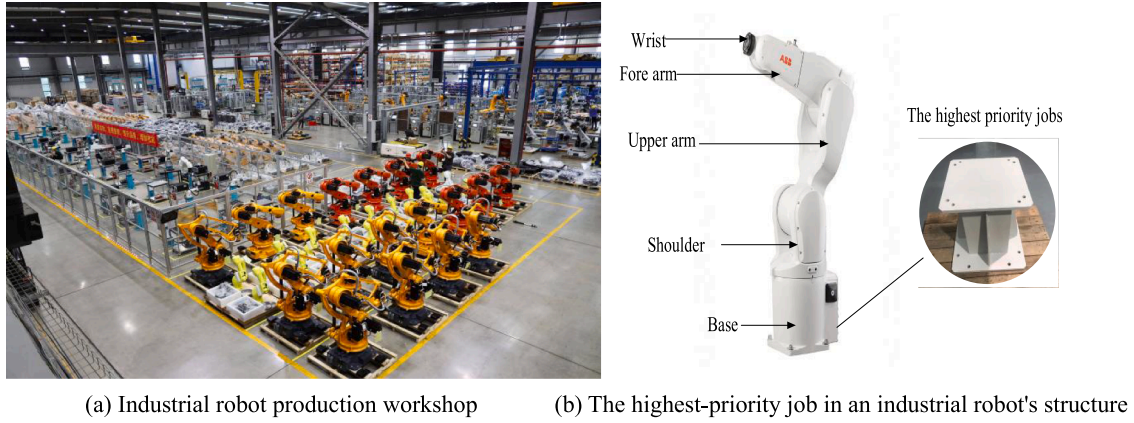


Fig. 1. Real-world context of DFJSP with job priority considerations.

progress directly influences the execution of downstream assembly processes and the delivery timeline of the overall equipment. Consequently, optimizing the production scheduling of high-priority tasks (such as the base) without affecting the overall project timeline has become an important issue that companies need to address to achieve efficient production.

Fig. 2 shows an example of the actual DFJSP model that takes job priority into account, in which there are a total of eight jobs and each job has five operations. From Fig. 2, we can see that the makespan of these two solutions is the same (equal to 80), and the completion times of jobs 1–8 in solution 1 are 46, 42, 60, 60, 67, 79, 74, 80, respectively, while those in solution 2 are 72, 72, 68, 80, 62, 56, 48, 43, respectively.

According to the preferences of the production manager, the evaluation results of these two approaches may be completely different.

- If the production manager only cares about the completion time, then the two solutions are the same; if the production manager only cares about the priority of job 5, the two solutions are also the same.
- If the production manager is concerned about the priorities of jobs 1, 2, and 3, solution 1 is better than solution 2.
- If the production manager is concerned about the priorities of jobs 6, 7, and 8, then solution 2 is better than solution 1.
- However, if the production manager cares about both the maximum completion time and the priorities of jobs 1, 5, and 6, there is no feasible solution.

Therefore, it is very urgent and important to propose an effective algorithm to solve the DFJSPJP problem, in order to help production managers make efficient decisions when considering problems with different job priorities.

In this paper, a multi-objective DFJSPJP model is proposed, aiming at minimizing the makespan, total energy consumption and the weighted average time of jobs with priority. As far as we know, this is the first time job priorities have been studied in related literature. MAs, which integrate evolutionary algorithms with local search, have demonstrated promising performance in solving multi-objective DFJSPs. Motivated by this, we propose a new NMA to address the DFJSPJP we proposed. In the NMA, a well-designed chromosome encoding mechanism (CEM) is utilized to establish a high-quality initial population by fully leveraging the information and structure of DFJSPJP. An effective local search method (LSO) is proposed and integrated into the genetic algorithm to enhance its convergence speed and fully exploit the solution space. Experimental results on a diverse set of up to 60 benchmark instances from literature validate the effectiveness of the proposed NMA and its components (CEM and LSO).

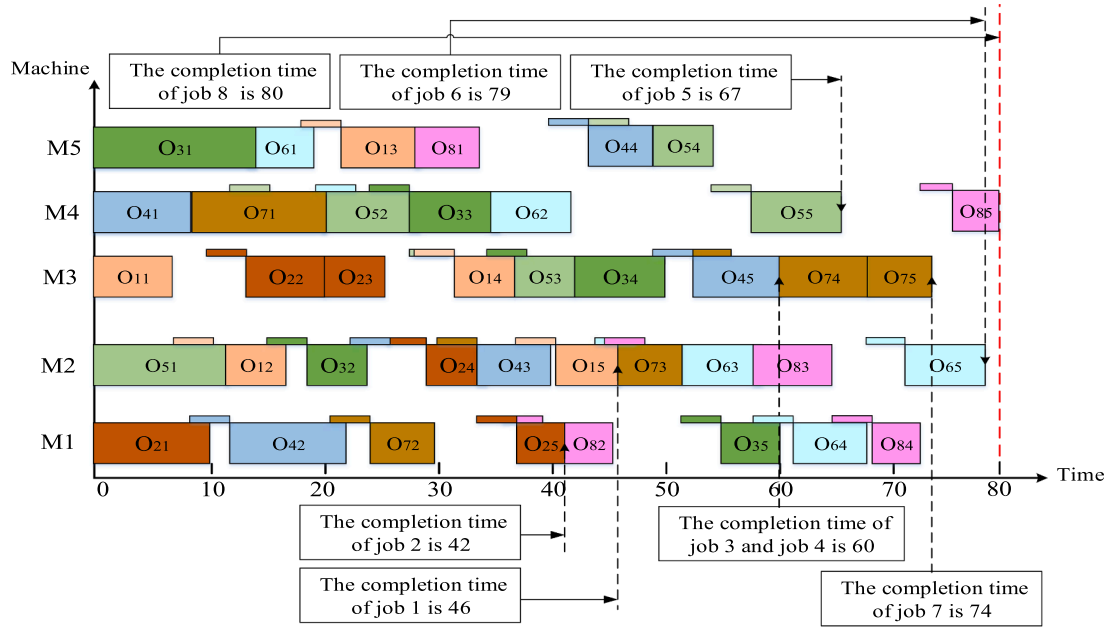
The rest of the article is organized as follows. Section 3 describes the mathematical model of the proposed DFJSPJP. Section 4 details our proposed NMA. Section 5 presents the experimental research and

discussion. Finally, conclusions are drawn in Section 6. The block diagram of the entire research is shown in Fig. 3.

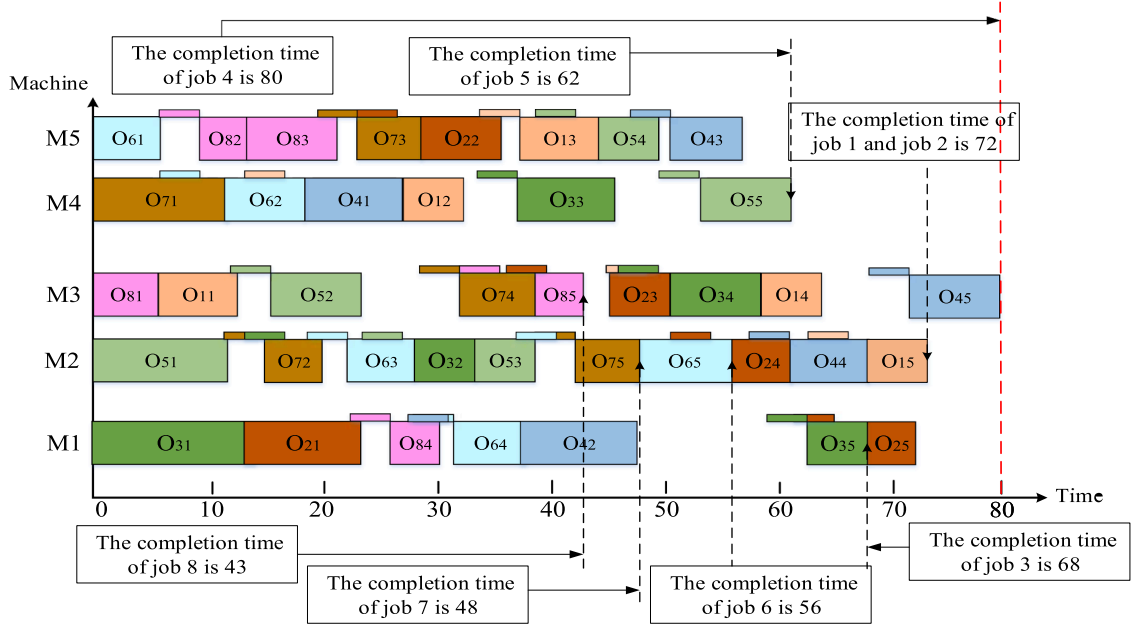
## 2. Literature review

### 2.1. DFJSP related work

De Giovanni and Pezzella [19] were pioneers in the study of DFJSP, where tasks were allocated to multiple flexible manufacturing units for concurrent processing. They developed an enhanced genetic algorithm (GA) incorporating greedy decoding and local search operators to tackle this challenge. Subsequently, a plethora of researchers have devised diverse approaches to address the DFJSP. Du et al. [20] set up a constraint programming model and developed a hybrid algorithm that integrates a distribution estimation algorithm and variable neighborhood search (VNS) to tackle the DFJSP, aiming to minimize the makespan and crane transportation. Recognizing the decentralized nature of the DFJSP model, Li et al. [21] introduced a hybrid tabu search algorithm with an optimization criterion and successfully applied it to a practical DFJSP scenario. Marzouki et al. [22] opted for a multi-agent system utilizing a meta-heuristic based on the forbidden search principle to optimize job assignments and factory scheduling, achieving commendable outcomes. Drawing inspiration from the collision principle in chemical reactions, Marzouki et al. [23] innovatively proposed a chemical reaction meta-heuristic algorithm for DFJSP optimization. Wu and Liu [24] formulated a mixed integer linear model for DFJSP and employed an enhanced differential evolutionary simulated annealing algorithm to pursue locally optimal solutions. Jiang et al. [25] introduced an improved multi-objective decomposition evolutionary algorithm to optimize energy-efficient DFJSP, aiming to minimize makespan and energy consumption. Chang and Liu [26] devised a hybrid GA incorporating a roulette encoding method to address job assignment challenges. Meng et al. [27] developed four mixed integer programming models and an efficient constraint programming model to solve the DFJSP, evaluating their efficacy through numerical experiments. They designed an energy regulation strategy to enhance algorithm performance. Furthermore, various enhanced hybrid GAs have been widely presented to tackle the complexities of DFJSP [28]. Wang et al. [18] proposed an improved adaptive hybrid algorithm (IAHA) to tackle the distributed flexible job shop scheduling problem (DFJSP) in the context of economic globalization and distributed manufacturing. The algorithm constructs a mathematical model for distributed factories, incorporates a dual-level encoding hybrid decoding rule, and employs an adaptive variable neighborhood search strategy to optimize cross-factory task allocation. Experimental results show that IAHA breaks multiple records and achieves optimal solutions in standard test cases involving 2–4 factories, highlighting its superior performance in solving DFJSP compared to existing methods.



(a) Solution 1



(b) Solution 2

Fig. 2. An example of a practical DFJSP considering job priority.

Gong et al. [29] made a breakthrough discovery that distributed workshops encompass not only factories and machines, but also workshops that vary across different factories. Subsequently, they introduced an effective memetic algorithm to tackle this challenge, aiming to minimize completion time and total energy consumption. Luo et al. [30] formulated a mixed integer linear programming (MILP) model for DFJSP with worker allocation taken into account. They devised an improved genetic algorithm (IMA) inspired by the framework of the second-generation non-dominated sorting genetic algorithm (NSGA-II), incorporating a streamlined two-level encoding approach, four heuristic decoding methods, and a logical adaptive neighborhood search operator. Wang et al. [31] proposed a feedback learning-based memetic algorithm (FLMA) aimed at minimizing both manufacturing time and total energy consumption, addressing the energy-aware distributed flexible job-shop scheduling problem with transportation constraints

(EDFJSP-T). Experiments were conducted using various types of instances, which demonstrated the effectiveness of the feedback learning mechanism and proved that FLMA outperforms existing algorithms in solving EDFJSP-T.

As mentioned above, existing research on DFJSP often considers some factors present in real manufacturing systems, such as worker flexibility, workshops, transportation, different types of factories Zheng et al. [32], Lu et al. [33], Xu et al. [34], and so on. However, the current studies on DFJSP have not taken into account the priority of job processing, which is widely prevalent in actual production environments where high-priority jobs need to be processed first. Therefore, research on DFJSP considering job priorities is of significant value in making the DFJSP model more aligned with real production scenarios.

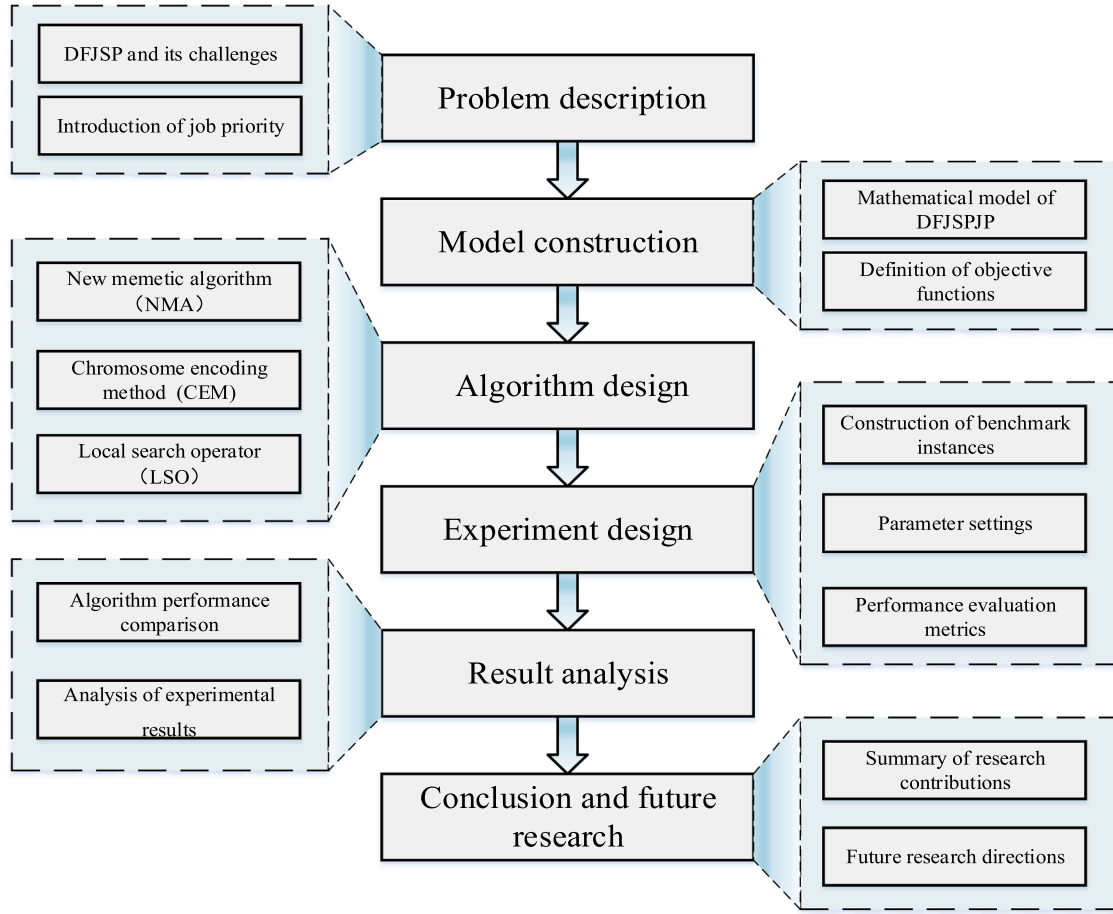


Fig. 3. Research block diagram.

## 2.2. Work related to job priority

In today's increasingly complex manufacturing environment, it is inevitable to consider the priority of job processing. Processing high-priority jobs as soon as possible will benefit the subsequent assembly and transportation work in the entire factory. Production scheduling with job priorities is very common and is a matter of great concern for production managers. Therefore, many researchers have studied various scheduling problems based on job priorities to achieve better production scheduling solutions. Li et al. [35] proposed a highly efficient hybrid self-adaptive differential evolution algorithm (HSDE) with heuristic strategies. They studied the flexible job shop scheduling problem with outsourcing and job priorities, achieving good results. Li et al. [36] proposed a dual deep Q-network-based co-evolutionary algorithm (D2QCE) to address the distributed heterogeneous hybrid flow shop scheduling problem with multi-priority jobs (DHHFSP-MPJ). By balancing global and local search, knowledge-driven neighborhood structures, and energy-saving strategies, D2QCE significantly reduces weighted tardiness and energy consumption. Experiments demonstrate that D2QCE outperforms existing algorithms on 20 benchmark instances and a real-world case of large-scale equipment manufacturing, providing an efficient solution for engineering practices that consider priority, delivery constraints, and resource optimization in scheduling decisions.

From the above literature, it is evident that priority, as an important research area in the academic community, has been highly valued by many researchers. However, previous studies on distributed flexible job shops did not consider the issue of job priorities, which is actually one of the most common requirements in modern production scenarios. For example, in a batch of job orders, there are several jobs that need to be

manufactured with priority. How should we handle such orders? Therefore, this paper innovatively studied the DFJSP considering job priorities, not only opening up a new research branch theoretically for future researchers, but also providing practical solutions for producers facing the DFJSP situations with job priority requirements.

## 3. Construction of DFJSPJP model

### 3.1. Problem description

The DFJSPJP model includes of  $n$  jobs  $J = \{J_1, J_2, \dots, J_n\}$  and  $n$  jobs are divided into  $n'$  priority levels with each job  $q_i$  operations; and these jobs should be distributed to be processed on different  $m$  machines in different  $w$  workshops of  $f$  different factories  $F = \{F_1, F_2, \dots, F_f\}$ . Each factory includes of  $w$  workshops  $W = \{W_1, W_2, \dots, W_w\}$ , and each workshop includes of  $m$  machines  $M = \{M_1, M_2, \dots, M_m\}$ . In this research, the time taken to complete each operation comprises its processing duration and transportation duration, while the overall energy usage involves the combined energy consumption for processing and transportation. The DFJSPJP model must address four sub-problems: 1) allocating factories; 2) assigning workshops; 3) distributing machines; 4) determining the sequence of operations.

The optimization objectives of the DFJSPJP model are as follows:

- (1) Minimize the makespan ( $f_1$ ).
- (2) Minimize the total energy consumption ( $f_2$ ).
- (3) Minimize the Weighted average time of job priority ( $f_3$ ).

The DFJSPJP model must meet the following constraints [11,37]:

**Table 1**  
Parameters, variables, and symbols of the DFJSPJP.

| <i>Parameters</i> |                                                                                                                                                                                                          |
|-------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| $i, h$            | Index of jobs                                                                                                                                                                                            |
| $j, g$            | Index of operations                                                                                                                                                                                      |
| $p$               | Index of factories                                                                                                                                                                                       |
| $w$               | Index of workshops                                                                                                                                                                                       |
| $m$               | Index of machines                                                                                                                                                                                        |
| $n'$              | Total number of jobs priorities, $n' \leq n$ ;                                                                                                                                                           |
| $a_i$             | The proportion of job of this level among all levels                                                                                                                                                     |
| $n$               | Total number of jobs                                                                                                                                                                                     |
| $n_i$             | Total number of operations of job $i$                                                                                                                                                                    |
| $n_f$             | Total number of factories                                                                                                                                                                                |
| $w_f$             | Total number of workshops in factory $f$                                                                                                                                                                 |
| $m_{fw}$          | Total number of machines in workshop $w$ of factory $f$                                                                                                                                                  |
| $I$               | Set of jobs, where $I = \{1, 2, \dots, n\}$                                                                                                                                                              |
| $Z_i$             | Set of operations of job $i$ , where $Z_i = \{1, 2, \dots, n_i\}$                                                                                                                                        |
| $ZZ_i$            | Set of the top $n_i - 1$ operations of job $i$ , $ZZ_i = \{1, 2, \dots, n_i - 1\}$                                                                                                                       |
| $O_{ij}$          | The $j$ -th operations of job $i$                                                                                                                                                                        |
| $F_i$             | Set of factories, $F_i = \{1, 2, \dots, n_f\}$                                                                                                                                                           |
| $W_f$             | Set of workshops in factory $f$ , $W_f = \{1, 2, \dots, w_f\}$                                                                                                                                           |
| $M_{ijfw}$        | Set of machines in workshop $w$ of factory $f$ that are eligible to process $O_{ij}$                                                                                                                     |
| $p$               | Index of positions                                                                                                                                                                                       |
| $p_{fwm}$         | Total number of positions of machine $m$ in workshop $w$ of factory $f$                                                                                                                                  |
| $P_{fwm}$         | Set of positions of machine $m$ in workshop $w$ of factory $f$ , $P_{fwm} = \{1, 2, \dots, p_{fwm}\}$                                                                                                    |
| $PP_{fwm}$        | Set of the top $p_{fwm} - 1$ positions of machine $m$ in workshop $w$ of factory $f$ , $PP_{fwm} = \{1, 2, \dots, p_{fwm} - 1\}$                                                                         |
| $PT_{ijfwm}$      | The processing time of $O_{ij}$ processed by machine $m$ in workshop $w$ of factory $f$                                                                                                                  |
| $T_m$             | The transportation time between two machines                                                                                                                                                             |
| $T_w$             | The transportation time between two workshops                                                                                                                                                            |
| $T_f$             | The transportation time between two factories                                                                                                                                                            |
| $PE_{fwm}$        | The processing power of machine $m$ in workshop $w$ of factory $f$                                                                                                                                       |
| $EU$              | The transportation power                                                                                                                                                                                 |
| $M$               | A large positive number                                                                                                                                                                                  |
| <b>Variables</b>  |                                                                                                                                                                                                          |
| $X_{ijfwp}$       | $X_{ijfwp} = \begin{cases} 1, & \text{if operation } O_{ij} \text{ is processed at position } p \text{ of machine } m \text{ in workshop } w \text{ of factory } f \\ 0, & \text{otherwise} \end{cases}$ |
| $X_{ijfmw}$       | $X_{ijfmw} = \begin{cases} 1, & \text{if operation } O_{ij} \text{ is assigned on machine } m \text{ of workshop } w \text{ of factory } f \\ 0, & \text{otherwise} \end{cases}$                         |
| $Y_{ijuv}$        | $Y_{ijuv} = \begin{cases} 1, & \text{if operation } O_{ij} \text{ is processed before operation } O_{uv}, i < u \\ 0, & \text{otherwise} \end{cases}$                                                    |
| $TM_{ij}$         | $TM_{ij} = \begin{cases} 1, & \text{if operation } O_{ij} \text{ is transported between two machines} \\ 0, & \text{otherwise} \end{cases}$                                                              |
| $TW_{ij}$         | $TW_{ij} = \begin{cases} 1, & \text{if operation } O_{ij} \text{ is transported between two workshops} \\ 0, & \text{otherwise} \end{cases}$                                                             |
| $TF_{ij}$         | $TF_{ij} = \begin{cases} 1, & \text{if operation } O_{ij} \text{ is transported between two factories} \\ 0, & \text{otherwise} \end{cases}$                                                             |
| $EM$              | Total transportation energy consumption among machines                                                                                                                                                   |
| $EW$              | Total transportation energy consumption among workshops                                                                                                                                                  |
| $EF$              | Total transportation energy consumption among factories                                                                                                                                                  |
| $EP$              | Total processing energy consumption of all operations                                                                                                                                                    |
| $ET$              | Total transportation energy consumption                                                                                                                                                                  |
| $ST_{ij}$         | The start time of $O_{ij}$                                                                                                                                                                               |
| $CT_{ij}$         | The completion time of $O_{ij}$                                                                                                                                                                          |
| $S_{fwp}$         | The start time of position $p$ of machine $m$ in workshop $w$ of factory $f$                                                                                                                             |
| $C_{max}$         | Makespan                                                                                                                                                                                                 |
| $E$               | Total energy consumption                                                                                                                                                                                 |
| $T_i$             | The total processing time of this level of jobs                                                                                                                                                          |

- (1) Cannot interrupt after the operation starts.
- (2) Each machine can only process a maximum of one operation at any time.
- (3) Jobs are independent of each other.
- (4) Machines are independent of each other
- (5) The movement time between operations and the setting time of the machine can be ignored.
- (6) Provide in advance the processing time corresponding to the job, operation, factory, workshop, and machine.
- (7) Provide in advance the transfer time and energy consumption between two machines, two workshops, and two factories.
- (8) Provide the total priority  $n'$  for this batch of jobs in advance.
- (9) Each operation must be processed continuously without interruption.

- (10) Any operation of any job cannot be processed until its previous process has been completed.

### 3.2. Parameters and variables

The parameters, symbols, and variables used in the DFJSPJP model are shown in Table 1.

### 3.3. Energy consumption model

The total energy consumption ( $E$ ) consists of two components: the overall energy consumption of machine processing ( $EP$ ) and the total energy consumption associated with transportation among machines, workshops, and factories ( $ET$ ) These are defined as follows:



- (1) The energy consumption ( $EP$ ) is produced by all machines in the processing state and can be computed using Equation (1).

$$EP = \sum_{i \in I} \sum_{j \in Z_i} \sum_{f \in F_i} \sum_{w \in W_f} \sum_{m \in M_{ijfw}} \sum_{p \in P_{fwm}} PT_{ijfwm} * X_{ijfwmp} * PE_{fwm} \quad (1)$$

- (2) The energy consumption ( $ET$ ) is generated by transport vehicles to move operations between various workshops, factories, and machines, and can be calculated using Equation (2).

$$ET = EF + EW + EM = \sum_{i \in I} \sum_{j \in Z_i} T_f * TF_{ij} * EU + \sum_{i \in I} \sum_{j \in Z_i} T_w * TW_{ij} * EU + \sum_{i \in I} \sum_{j \in Z_i} T_m * TM_{ij} * EU \quad (2)$$

### 3.4. Mathematical model of DFSPJP

The mathematical model of DFJSPJP is based on the above parameters and energy consumption model.

#### Objectives:

- (1) Minimizing the makespan ( $f1$ ).

$$f1 = \min C_{max} \quad (3)$$

- (2) Minimizing the total energy consumption ( $f2$ ).

$$f2 = \min E = \min(EP + ET) \quad (4)$$

- (3) Minimize the Weighted average time of job priority ( $f3$ ).

$$f3 = \sum_{i=1}^n T_i * a_i \quad (5)$$

#### Subject to:

$$\sum_{f \in F_i} \sum_{w \in W_f} \sum_{m \in M_{ijfw}} \sum_{p \in P_{fwm}} X_{ijfwmp} = 1, \forall i \in I, j \in Z_i \quad (6)$$

$$\sum_{i \in I} \sum_{j \in Z_i} \sum_{f \in F_i} \sum_{w \in W_f} X_{ijfwmp} \geq \sum_{i \in I} \sum_{j \in Z_i} \sum_{f \in F_i} \sum_{w \in W_f} X_{ijfwmp+1}, \forall m \in M_{ijfw}, p \in PP_{fwm} \quad (7)$$

$$ST_{i(j+1)} \geq ST_{ij} + \sum_{f \in F_i} \sum_{w \in W_f} \sum_{m \in M_{ijfw}} \sum_{p \in P_{fwm}} (PT_{ijfwm} + T_f * TF_{ij} + T_w * TW_{ij} + T_m * TM_{ij}) * X_{ijfwmp},$$

$$\forall i \in I, j \in ZZ_i, f \in F_i, w \in W_f, m \in M_{ijfw}, p \in P_{fwm} \quad (8)$$

$$\sum_{i \in I} \sum_{j \in Z_i} X_{ijfwmp} \leq 1, \forall f \in F_i, w \in W_f, m \in M_{ijfw}, p \in PP_{fwm} \quad (9)$$

$$ST_{ij} + PT_{ijfwm} * X_{ijfwm} \leq ST_{uv} + M(3 - Y_{ijuv} - X_{ijfwm} - X_{uvfwm}),$$

$$\forall i \in I, u \in I, i < u, j \in Z_i, v \in Z_u, f \in F, m \in M_{ijfw} \cap M_{uvfw} \quad (10)$$

$$ST_{uv} + PT_{uvfwm} * X_{uvfwm} \leq ST_{ij} + M(2 + Y_{ijuv} - X_{ijfwm} - X_{uvfwm}),$$

$$\forall i \in I, u \in I, i < u, j \in Z_i, v \in Z_u, f \in F, m \in M_{ijfw} \cap M_{uvfw} \quad (11)$$

**Table 2**

An example of DFJSPJP.

| Jobs  | Operations | $F_1$     |           |           | $F_2$     |           |           |
|-------|------------|-----------|-----------|-----------|-----------|-----------|-----------|
|       |            | $W_{11}$  |           | $W_{12}$  | $W_{21}$  |           |           |
|       |            | $M_{111}$ | $M_{112}$ | $M_{121}$ | $M_{211}$ | $M_{212}$ | $M_{213}$ |
| $J_1$ | $O_{11}$   | 5         | —         | 4         | —         | —         | 5         |
|       | $O_{12}$   | —         | 5         | —         | 4         | —         | 7         |
| $J_2$ | $O_{21}$   | —         | 4         | 7         | 6         | —         | —         |
|       | $O_{22}$   | 3         | —         | —         | —         | 5         | —         |
| $J_3$ | $O_{23}$   | —         | 5         | 6         | 9         | —         | —         |
|       | $O_{31}$   | —         | —         | —         | —         | 7         | 9         |
|       | $O_{32}$   | 8         | —         | 5         | —         | —         | 6         |
| $J_4$ | $O_{33}$   | 4         | —         | —         | —         | 3         | —         |
|       | $O_{41}$   | —         | 6         | —         | 5         | —         | 7         |
|       | $O_{42}$   | 9         | —         | 8         | —         | —         | —         |
|       | $O_{43}$   | 5         | —         | 5         | 3         | 4         | —         |

$$S_{fwmp} \geq ST_{ij} - (1 - X_{ijfwmp}) * M, \forall i \in I, j \in Z_i, f \in F_i, w \in W_f, m \in M_{ijfw}, p \in P_{fwm} \quad (12)$$

$$S_{fwmp} \leq ST_{ij} + (1 - X_{ijfwmp}) * M, \forall i \in I, j \in Z_i, f \in F_i, w \in W_f, m \in M_{ijfw}, p \in P_{fwm} \quad (13)$$

$$CT_{ij} \geq ST_{ij} + \sum_{f \in F_i} \sum_{w \in W_f} \sum_{m \in M_{ijfw}} \sum_{p \in P_{fwm}} (PT_{ijfwm} + T_f * TF_{ij} + T_w * TW_{ij} + T_m * TM_{ij}) * X_{ijfwmp},$$

$$\forall i \in I, j \in Z_i, f \in F_i, w \in W_f, m \in M_{ijfw}, p \in P_{fwm} \quad (14)$$

$$ST_{i(j+1)} \geq CT_{ij}, \forall i \in I, j \in ZZ_i \quad (15)$$

$$C_{max} \geq ST_{in_i} + \sum_{f \in F_i} \sum_{w \in W_f} \sum_{m \in M_{ijfw}} \sum_{p \in P_{fwm}} (PT_{in_ifwm} + T_f * TF_{in_i} + T_w * TW_{in_i} + T_m * TM_{in_i}) * X_{in_ifwmp},$$

$$\forall i \in I, j \in Z_i, f \in F_i, w \in W_f, m \in M_{ijfw}, p \in P_{fwm} \quad (16)$$

$$0 \leq (TF_{ij} + TW_{ij} + TM_{ij}) \leq 1, \forall i \in I, j \in Z_i \quad (17)$$

$$ST_{ij} \geq 0, \forall i \in I, j \in Z_i \# \quad (18)$$

$$CT_{ij} \geq 0, \forall i \in I, j \in Z_i \# \quad (19)$$

$$S_{fwmp} \geq 0, \forall f \in F_i, w \in W_f, m \in M_{ijfw}, p \in P_{fwm} \quad (20)$$

$$CT_{ij} - ST_{ij} = PT_{ijfwm}, \forall i \in I, j \in Z_i, f \in F_i, w \in W_f, m \in M_{ijfw} \quad (21)$$

Eqs. (3) to (5) is used to calculate the optimization objectives for  $f1$ ,  $f2$  and  $f3$ , respectively. Eq. (6) ensures that each operation must be assigned to a position on a machine. Eq. (7) stipulates that each machining position on the machine must be assigned to the operation in sequence. Eq. (8) ensures that the processing order constraint between each job operation is. Eqs. (9) - (11) ensure that each machine can arrange up to one operation at each position to avoid overlapping operations. Eqs. (12) - (13) describe the correlation between the starting time of each position on the machine and the processing duration of the assigned operation. Eqs. (14) - (15) establish the connection between the completion time and the starting time of the operation. Eq. (16) sets the objective of minimizing makespan, ensuring it is equal to or greater than the completion time of the last operation in any job. Eq. (17) restricts each operation to a maximum of one transportation between two

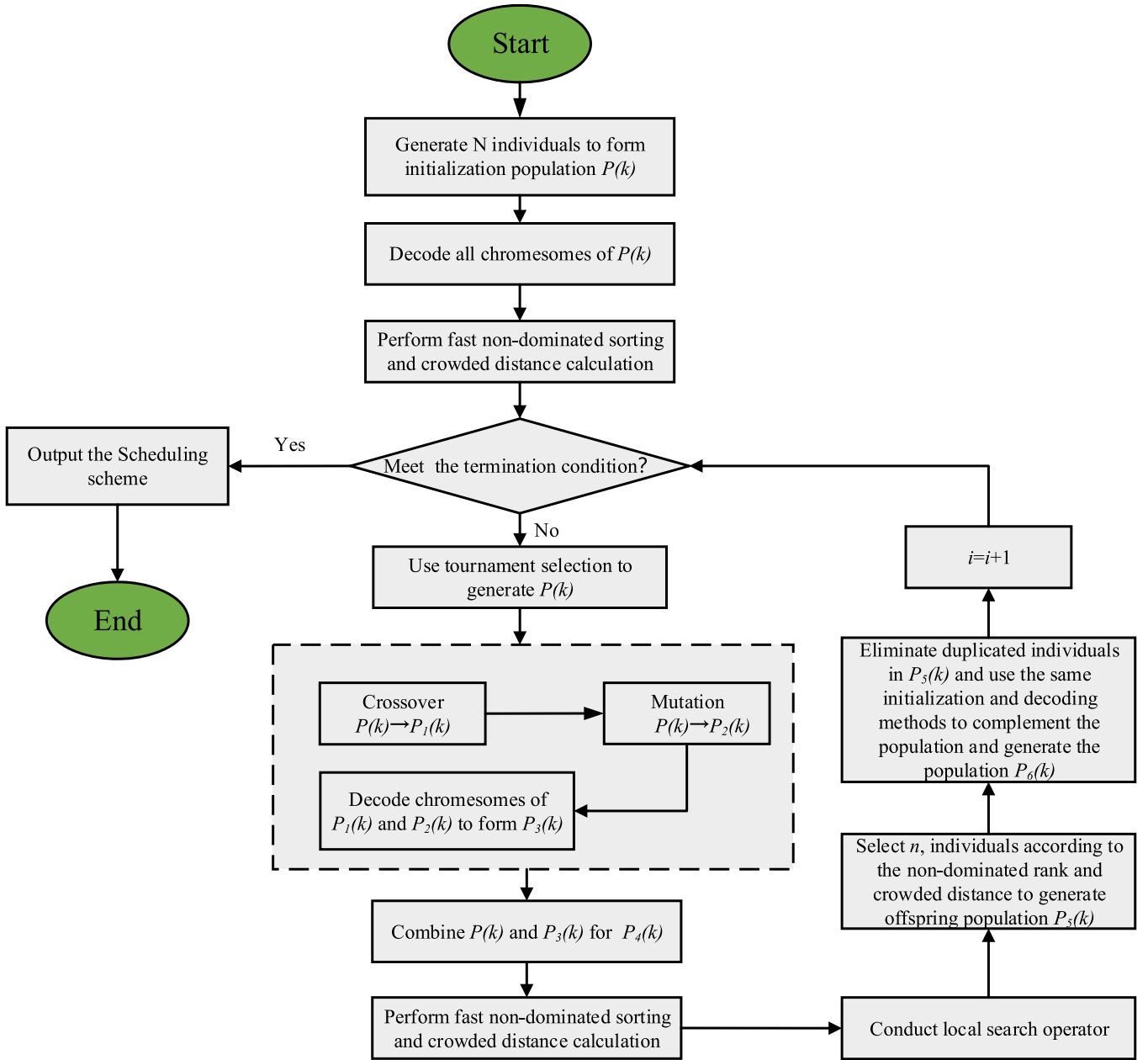


Fig. 4. The flow chart of the proposed NMA.

factories, workshops, or machines. Eqs. (18) - (20) guarantee that the starting time, completion time, and start time of all subsequent operations at every position on the machine are equal to or greater than 0. As per Eq. (21), each operation cannot be interrupted once it has started.

### 3.5. Model example

Table 2 presents an illustration of the DFJSPJP model, detailing processing information for all job operations. In this example, there are two different factories ( $F_1$  and  $F_2$ ),  $F_1$  has two workshops ( $W_{11}$  and  $W_{12}$ ), and  $F_2$  has one workshop ( $W_{21}$ ).  $W_{11}$  includes  $M_{111}$  and  $M_{112}$ ,  $W_{12}$  includes  $M_{121}$  and  $M_{122}$ , and  $W_{21}$  includes  $M_{211}$  and  $M_{212}$ . There are a total of 4 job positions ( $J_1, J_2, J_3, J_4$ ). Job 1 has two operations:  $J_1$  ( $O_{11}, O_{12}$ ), and each other job has three operations:  $J_2$  ( $O_{21}, O_{22}, O_{23}$ ), ...,  $J_4$  ( $O_{41}, O_{42}, O_{43}$ ). Each operation can be processed by only one of the machines in the designated factory and workshop.

The numbers in the table represent processing times. For example,

processing  $O_{21}$  at machine  $M_{121}$  in workshop  $W_{12}$  of factory  $F_1$  takes 7 unit processing time, and it cannot be processed at machine  $M_{111}$  in workshop  $W_{11}$  of factory  $F_1$ .

## 4. The proposed NMA

As we all know, the NSGA-II is one of the most popular evolutionary algorithms to solve the production scheduling problems [30,38,39], in which a selection strategy is used to choose the excellent individuals for the population, and the non-dominant sorting and crowding distance operators are designed to deal with the optimization objectives. In recent decades, the MA has also been widely used to solve the scheduling problems based on the framework of NSGA-II [40,41], in which an efficient local search operator (LSO) is integrated [42–44]. Here, we propose an NMA to solve the DFJSPJP, in which an efficient LSO is combined to expend the solution space and increase the convergence speed while retaining the advantages of both NSGA-II and MA. The

|    | Job 1    |          | Job 2    |          |          | Job 3    |          |          | Job 4    |          |          |
|----|----------|----------|----------|----------|----------|----------|----------|----------|----------|----------|----------|
|    | $O_{11}$ | $O_{12}$ | $O_{21}$ | $O_{22}$ | $O_{23}$ | $O_{31}$ | $O_{32}$ | $O_{33}$ | $O_{41}$ | $O_{42}$ | $O_{43}$ |
| FS | 2        | 2        | 1        | 2        | 1        | 2        | 2        | 2        | 1        | 2        | 1        |
| WS | 1        | 1        | 1        | 1        | 2        | 1        | 1        | 1        | 1        | 1        | 1        |
| MS | 3        | 1        | 2        | 2        | 1        | 2        | 3        | 2        | 2        | 1        | 1        |
| OS |          |          |          |          |          |          |          |          |          |          |          |
|    | $O_{21}$ | $O_{31}$ | $O_{11}$ | $O_{32}$ | $O_{41}$ | $O_{12}$ | $O_{22}$ | $O_{23}$ | $O_{42}$ | $O_{33}$ | $O_{43}$ |
| OS | 2        | 3        | 1        | 3        | 4        | 1        | 2        | 2        | 4        | 3        | 4        |
| PS | 3        | 2        | 1        | 2        | 3        | 3        | 3        | 3        | 3        | 2        | 3        |

Fig. 5. An example of the five-layer encoding method.

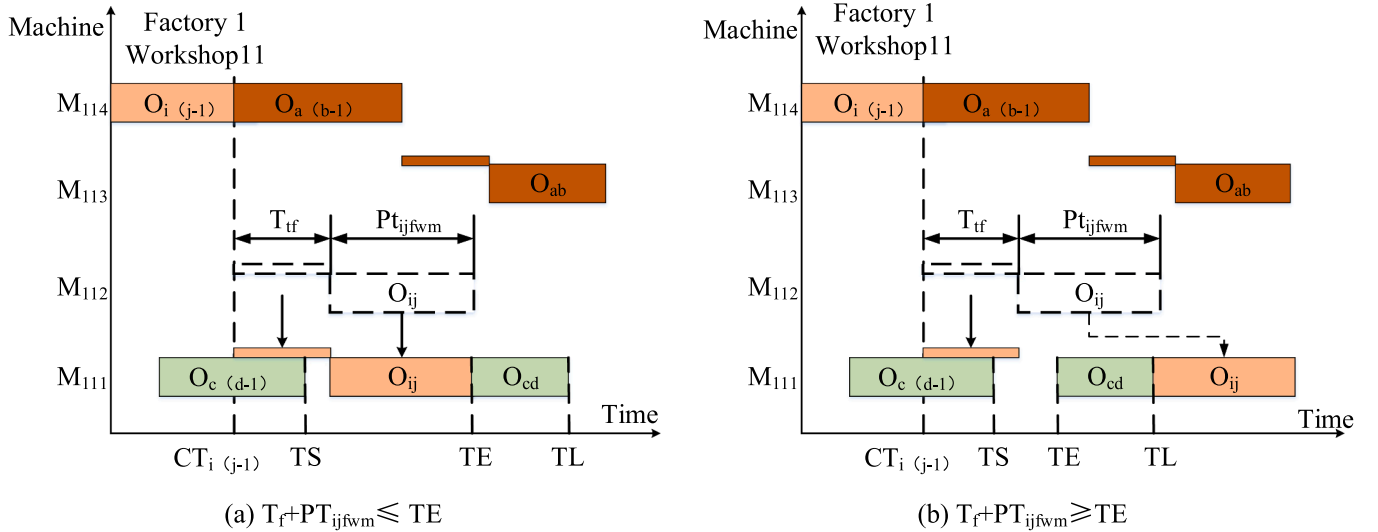


Fig. 6. Insert decoding method of the DFJSPJP.

framework of the proposed NMA is shown in Fig. 4.

#### 4.1. Encoding and decoding

This article designs a five-type encoding method (CEM) to represent FS, WS, MS, OS, and PS vectors respectively. Based on the data in Table 2, an example of an encoding is shown in Fig. 5. In the OS vector ([2,3,1,3,4,1,2,2, 4, 3, 4]), it represents that job 1 has two operations (encoded as two "1"s), job 2 has three operations (encoded as three "2"s), job 3 has three operations (encoded as three "3"s), and job 4 has two operations (encoded as two "4"s). In OS, the  $t$ -th occurrence of an integer represents the  $t$ -th operation of Job  $i$ .

The FS vector ([2,2,1,2,1,2,2, 1, 2, 1]) consists of four parts (because there are 4 jobs), each part represents the factory selected for the corresponding operation of a job. For example, the first part includes the numbers "2" and "2". The first number "2" indicates that factory 2 is selected to process job 1's operation 1 ( $O_{11}$ ); the second number "2" indicates that factory 2 is selected to process job 1's operation 2 ( $O_{12}$ ).

The WS vector ([1,1,1,2,1,2,1,1, 1, 1, 1]) also consists of four parts, each part represents the workshop selected for the corresponding operation of a job. For example, the first part includes the numbers "1" and "1". The first number "1" indicates that workshop 1 of factory 2 is selected to process job 1's operation 1 ( $O_{11}$ ); the second number "1" indicates that workshop 1 of factory 2 is selected to process job 1's operation 2 ( $O_{12}$ ).

The MS vector ([3,1,2,2,1,2,3,2, 2, 1, 1]) also consists of four parts, each part represents the machine selected for the corresponding operation of a job. For example, the first part includes the numbers "3" and "1". The first number "3" indicates that machine 3 in workshop 1 of factory 2 is selected to process job 1's operation 1 ( $O_{11}$ ); the second number "1" indicates that machine 1 in workshop 1 of factory 2 is selected to process job 1's operation 2 ( $O_{12}$ ).

The PS vector ([3,2,1,2,3,1,3,3, 3, 2, 3]) represents the job priorities, where the first number "3" indicates that job 3 has a priority of 3, and the second number "2" indicates that job 2 has a priority of 2.

As depicted in Fig. 6, an enhanced idle time insertion decoding



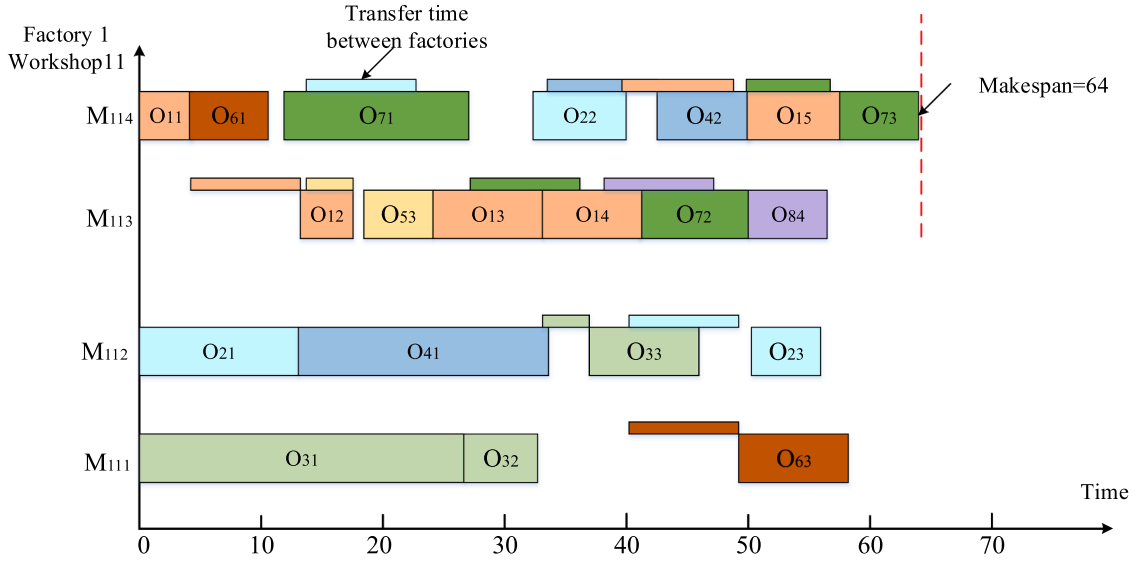


Fig. 7. A decoded gantt chart of the DFJSPJP.

technique [45] is employed to decode the chromosomes, enabling each operation to be processed at the earliest opportunity and maximizing the utilization of machine idle time. The detailed steps are outlined below:

**Step 1.** Extract a gene from the OS gene segment, denoted as  $O_{ij}$ . Obtain the processing factory  $f$ , workshop  $w$ , and machine  $m$  for  $O_{ij}$  from the respective chromosome, and then ascertain the processing time  $PT_{ijfwm}$  and transportation time  $T_{tf}$  for the operation.

**Step 2.** Identify all idle intervals  $[TS, TE]$  on  $M_l$  in workshop 21 of factory 2, where  $TS$  and  $TE$  indicate the start time and completion time of the interval, respectively. Calculate the earliest processing time  $T_p$  for  $O_{ij}$  using Eq. (24). If  $j \leq 1$ ,  $T_p=0$ .

$$T_p = \max\{CT_{i(j-1)} + T_{tf}, TS\} \quad (24)$$

**Step 3.** Utilize Eq. (25) is used to decide if  $O_{ij}$  can be inserted into the available interval. If affirmative, place it within this interval; if negative, designate the insertable start time of  $O_{ij}$  as  $T_b$  based on Eq. (26). Where  $T_{lm}$  denotes the current completion time of the last operation on  $M_l$ . The detailed process of these two insertion methods is illustrated in Fig. 6.

$$T_p + PT_{ijfwm} \leq TE \quad (25)$$

$$T_b = \max\{CT_{i(j-1)} + T_{tf}, T_{lm}\} \quad (26)$$

**Step 4.** Decode each operation sequentially using the aforementioned method until all operations are processed.

When the chromosome is decoded, each operation is processed as early as possible according to priority, operation constraints, and

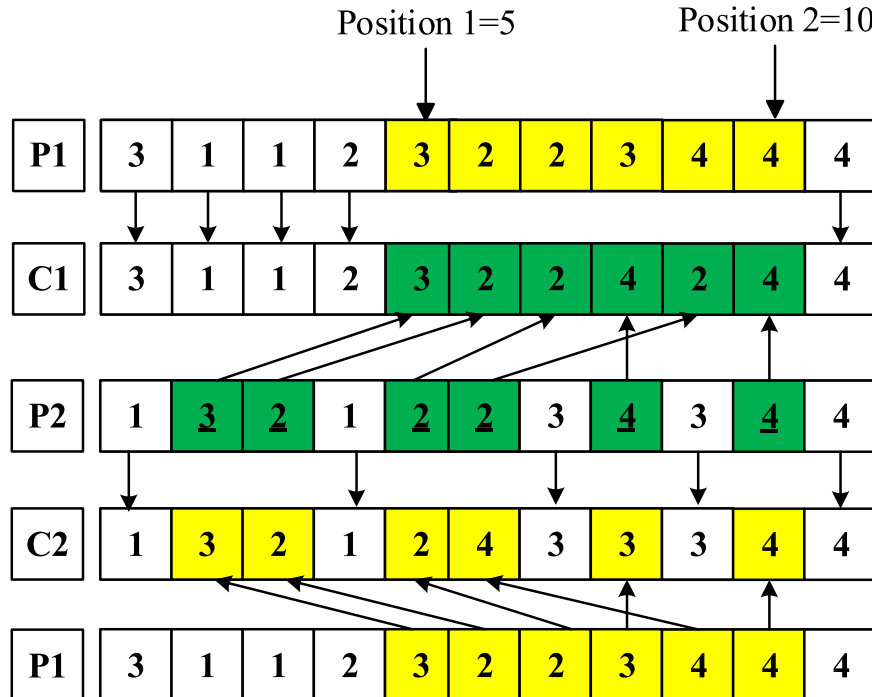


Fig. 8. A diagram of the REC.

machine constraints, ensuring that the schedule generated by this decoding method is an active schedule. Fig. 7 shows the Gantt chart obtained after decoding the chromosome.

#### 4.2. Population initialization

A novel load balancing initialization rule has been developed based on the proposed CEM encoding operator with five levels. Initially, job priorities are randomly assigned and the job operations are sorted. The key concept is to prioritize operations of jobs with higher priority, assigning them to the same factory, workshop, and machine as the preceding operation to minimize inter-machine, inter-workshop, and inter-factory transportation. Alternatively, if needed, cumulative processing times for machines, workshops, and factories are calculated. Subsequently, machines, workshops, and factories are selected for each operation in sequence based on the minimum cumulative processing time. The specific steps are outlined below, with the core pseudocode of the initialization method detailed in Algorithm 1.

**Step 1.** Set the total job level as "pri" and randomly assign priorities to the jobs.

**Step 2.** There are a total of  $n$  jobs, pri level, set the range of random number is  $n + (pri-1) + \dots + 1$ , if the selected random number  $r$  belongs to  $1 \sim n$ , then the operations of the  $r$ -rd job is sorted, if  $r$  is  $> n$ , according to the number obtained based on  $r-n$ , judge the job level  $t$ , randomly select a job with a level  $t$  for sorting, until all operations are arranged. As mentioned above, jobs with high priority will be scheduled for processing first to optimize the weighted average time of job priorities. For example, there are 10 jobs with 4 levels and level 1 is the highest. Set the range of random numbers to  $10+3+2+1$ , the random number in  $1 \sim 10$  represents the job  $1 \sim 10$ , in  $11 \sim 13$  select the job with level 1,  $14 \sim 15$  select the job with level 2, and 16 select the job with level 3. In this way, jobs with higher priorities can be sequenced earlier.

**Step 3.** If the random number is  $1 \sim 10$ , then assign the operation of the corresponding job according to the random number. If a gene of the chromosome (representing an operation) represents the first operation of a job, allocate a factory, workshop, and machine to minimize the cumulative time for this operation; otherwise, proceed to step 4.

**Step 4.** If a job shares the same factory, workshop, and machine as a previous job, assign the same factory, workshop, and machine to this job, otherwise, assign the factory, workshop, and machine with the minimum cumulative time.

**Algorithm 1:** [pop\_st]=Initilization(prio, job-num, pop-size, all-oper, ope-set).

```
% prio: The total priority of jobs
% pop-size: Population size
% job-num: The total number of jobs
% all-oper: The sum of operation for all jobs
% ope-set: The set of the number of operations for each job
1: Pop_st ← cell (1: pop-size, 1);
2: For j = 1 to pop-size Do
3: num ← 0; Chrom ← zeros (6, all_oer);
4: For i = 1 to job-num Do
5: oper ← ope-set (2, i); Chrom(1, num+1:num+oper) ← ones (1, oper)*i; num ← num+oper;
6: End For
7: For r = 1 to job-num Do
8: pri-set(r)=randperm (prio);
9: HA=randperm (job-num); chrom (HA)=pri-set(1:job-num);
10: Chrom (5, :) ← Randomly arrange the priority in Chrom (5, :);
11: For j = 1:all-oper Do
12: P=unidrnd (job-num+sum(1:pri-1));
13: If the number of P is bigger than the number of P Then
14: n = P-job-num; Randomly select a job with a priority less than n, named BA;
15: Chrom (4, :) ← Randomly arrange the BA in Chrom (4, :);
```

(continued on next column)

(continued)

```
16: Else
17: Chrom (4, :) ← Randomly arrange the gene in Chrom (4, :);
18: End For
19: For p = 1 to all-oper Do
20: The processing status of each operation are saved into Chrom (7, :);
21: End For
22: For z = 1 to all-oper Do
23: Read genes in Chrom one by one from left to right;
24: If the current operation is the first operation of the jobs Then
25: The factory fa-rank with the shortest cumulative processing time was randomly
    selected from the available factories
    set avai-fa to process the current operation;
26: The workshop wo-rank with the shortest cumulative processing time was randomly
    selected from the available workshop set avai-wo to process the current operation;
27: The machine ma-rank with the shortest cumulative processing time was randomly
    selected from the available machine set avai-ma to process the current operation;
28: Record the processing time pro-time and processing cost pro-cost of this operation
    in the fa-rank, wo-rank and ma-rank;
29: Chrom (1:3, z) ← [fa-rank, wo-rank, ma-rank];
30: Chrom (5:6, z) ← [pro-time, pro-cost];
31: Else
32: [Pre-fa, Pre-wo, Pre-ma] ← [Chrom(1, z-1), Chrom(2, z-1), Chrom(3, z-1)];
33: If Pre-fa==avai-fa Then
34: If Pre-wo==avai-wo Then
35: If Pre-ma==avai-ma Then
36: ma-fank ← Pre-ma; wo-rank ← Pre-wo; fa-rank ← Pre-fa;
37: Else
38: Select an available machine ma1_rank from avai-ma at random;
39: ma-rank ← ma1_rank; wo-rank ← Pre-wo; fa-rank ← Pre-fa;
40: End If
41: Else
42: Select an available workshop wo1_rank from avai-wo at random;
43: Select an available machine ma1_rank from avai-ma at random;
44: ma-rank ← ma1_rank; wo-rank ← wo1_rank; fa-rank ← Pre-fa;
45: End If
46: Else
47: Select an available factory fa1_rank from avai-fa at random;
48: Select an available workshop wo1_rank from avai-wo at random;
49: Select an available machine ma1_rank from avai-ma at random;
50: ma-rank ← ma1_rank; wo-rank ← wo1_rank; fa-rank ← fa1_rank;
51: End If
52: Chrom (1:3, z) ← [fa-rank, wo-rank, ma-rank];
53: Chrom (5:6, z) ← [pro-time, pro-cost];
54: End If
55: End For
56: Pop-st(j) ← Chrom;
57: End For
58: Return (Pop-st)
```

#### 4.3. Crossover operator

The crossover operator aims to enhance solutions by exchanging favorable genes between two chromosomes from the parental population. To improve the quality of the offspring population, this research introduces a random exchange crossover method (REC) into NMA. An illustration of REC is provided in Fig. 8, with  $P1$  and  $P2$  representing parents, and  $C1$  and  $C2$  representing offspring. The specific operational steps are outlined below:

**Step 1.** Randomly choose two positive integers ranging from 0 to the total number of operations in  $P1$ , identified as positions 1 and 2. In the illustration, position 1 = 5, position 2 = 10.

**Step 2.** Extract the gene segment between positions 1 and 2 in  $P1$ , [2,3,2,2,3,4,4]; identify the operations corresponding to this gene segment as  $O_{21}$ ,  $O_{32}$ ,  $O_{22}$ ,  $O_{23}$ ,  $O_{33}$ ,  $O_{41}$ ,  $O_{42}$ ,  $O_{43}$ .

**Step 3.** In  $P2$ , find the genes that are the same as [2,3,2,2,3,4,4] from left to right, meaning find the genes in  $P2$  that are the same as [3,2,2,2,3,4,4,4]; the corresponding operations in  $P2$  for [3,2,2,2,3,4,4,4] are  $O_{31}$ ,  $O_{21}$ ,  $O_{22}$ ,  $O_{23}$ ,  $O_{32}$ ,  $O_{41}$ ,  $O_{42}$ ,  $O_{43}$ .

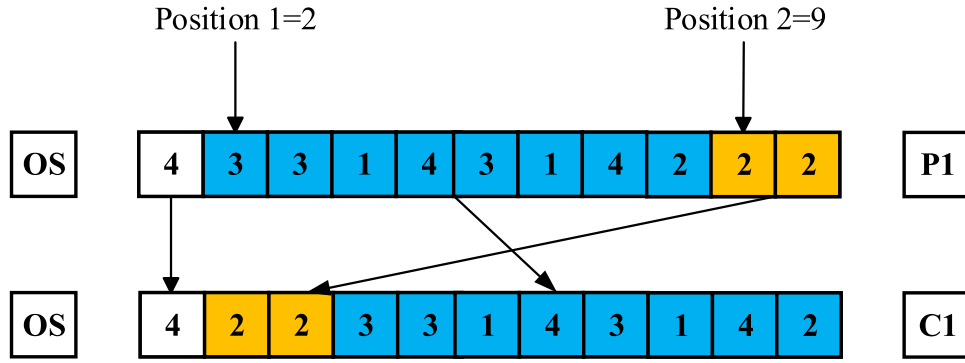


Fig. 9. A diagram of the RAM.

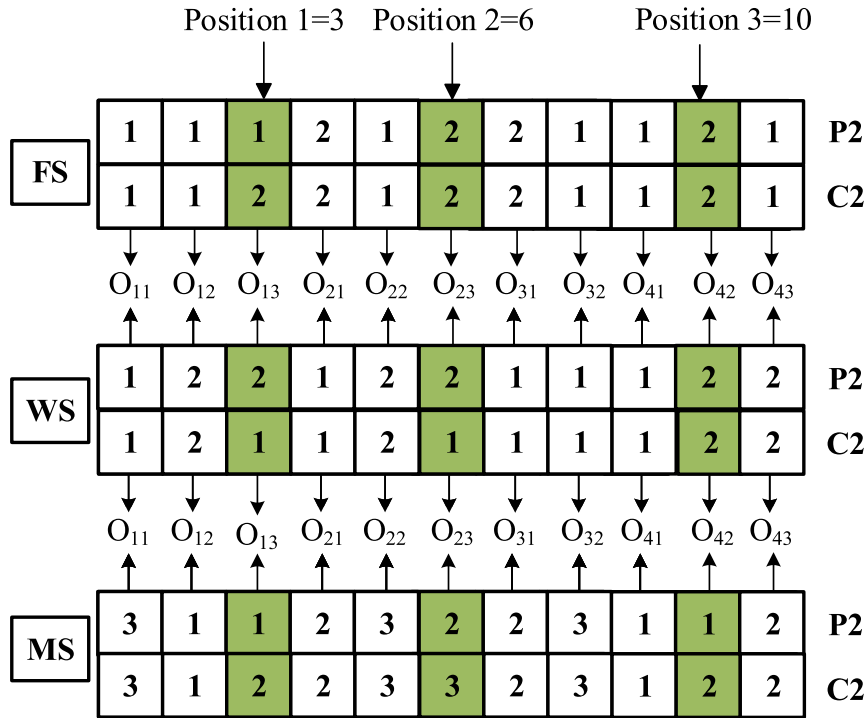


Fig. 10. A diagram of the TPM.

**Step 4.** Randomly arrange the genes [3,2,2,2,3,4,4,4] from P2 between positions 1 and 2 in C1 (in this example, the random order is 4, 2, 4, 4, 3, 2, 2, 3); the other genes from P1 are inherited to the vacant positions in C1.

C2 can be obtained using the same method as C1.

#### 4.4. Mutation operators

The mutation operator is primarily aimed at preserving the evolutionary population's diversity and preventing convergence to local optima. In this system, a random permutation mutation operator (RAM) is proposed, while three-position mutation operators (TPM) are developed for FS, WS, and MS to uphold population diversity and avert local optima. Figs. 9 and 10 illustrate RAM and TPM, with P1 and P2 denoting parents, and C1 and C2 representing offspring. The specifics of RAM are delineated as follows:

**Step 1:** Randomly pick two positive integers within the range of 0 to the total number of operations in P1, identified as positions 1 and 2. In the illustration, position 1 = 2, position 2 = 9.

**Step 2:** Retrieve the gene set between positions 1 and 2 in P1, T2=

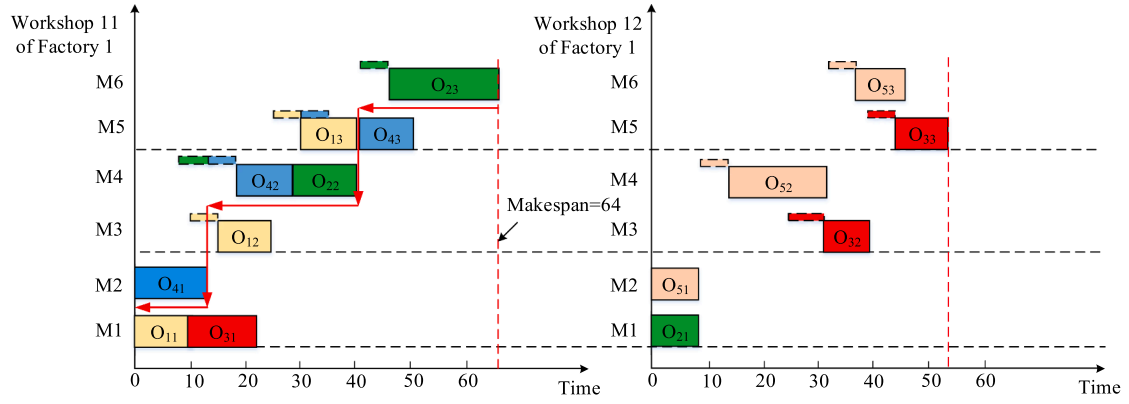
[3,3,1,4,3,1,4,2]. Extract the gene set between the initial position and P1, T1= [4], and the gene set between P2 and the final position, T3= [2,2]. Subsequently, arrange them in random sequence in the offspring diagram as (T1, T3, T2), producing the offspring individual C1 diagram as [4,2,2,3,3,1,4,3,1,4,2].

The TPM details are outlined as follows:

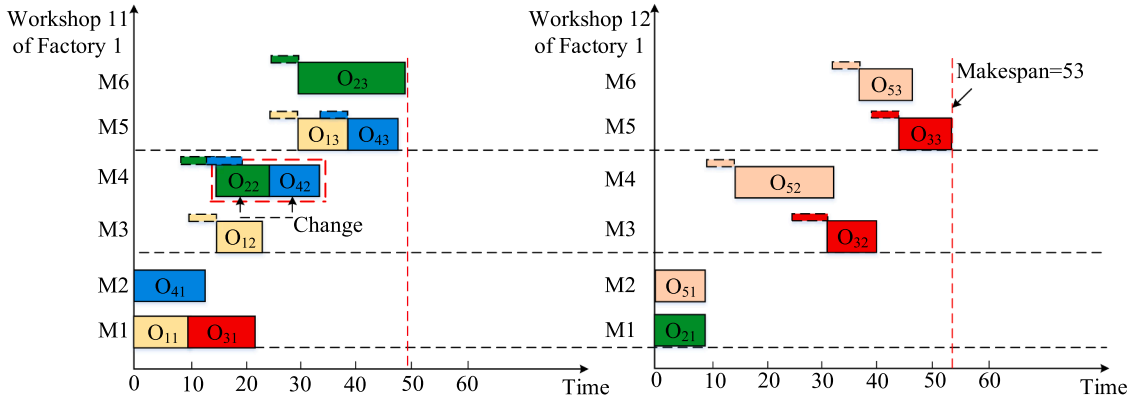
**Step 1.** Randomly select three positions (in this instance, positions 3, 6, and 10 from P2 of FS, WS, and MS are chosen).

**Step 2.** Determine the adaptability of the operations at the three selected positions for factory modifications. If feasible, adjust the factory, workshop, and machine accordingly (as shown in Fig. 10, for instance, at the third position, change O<sub>13</sub> from machine 1 in workshop 2 of factory 1 to machine 2 in workshop 1 of factory 2); if not feasible, retain the factories unchanged.

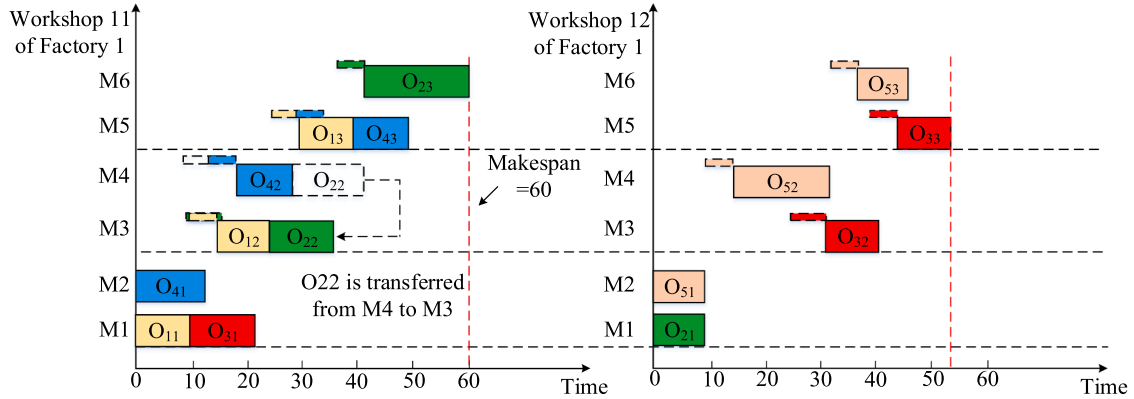
**Step 3.** Evaluate if workshop modifications are viable for the operations at the selected three positions. If feasible, make changes to the workshop and machine (as illustrated in Fig. 10, at the sixth position, switch O<sub>23</sub> from machine 2 in workshop 2 to machine 3 in workshop 1);



(a) An example of the initial critical path



(b) The result of OEM



(c) The result of MEM

Fig. 11. An example of the proposed LSO.

if not viable, keep the workshops unchanged.

**Step 4.** Assess the possibility of machine modifications for the operations at the three selected positions. If possible, carry out alterations to the machines (shown in Fig. 10, changing  $O_{42}$  from machine 1 to machine 2); if not possible, maintain the machines as they are.

By following the above steps, C2 can be generated.

#### 4.5. Local search operator

In order to sustain population diversity and prevent getting stuck in local optima, an efficient local search operator (LSO) has been introduced. LSO comprises two parts: the operation exchange method (OEM)

and the machine exchange method (MEM). The core procedure is depicted in Fig. 11 and the critical path in Fig. 11(a) usually represents the longest route from the initial node to the terminal node in the scheduling framework, which equals the makespan.

For OEM, multiple positions are randomly generated on the critical operations, and then the priority levels of these jobs are evaluated to prioritize higher-priority jobs at the front. For MEM, the approach involves finding the machine with the shortest processing time on critical operations, and then shortening the makespan of the solution by exchanging machines and rearranging them.

An example of OEM is shown in Fig. 11(b) as follows:

**Step 1.** Randomly find two adjacent operations next to each other on the critical blocks located on the critical path at the same stage. In the

**Table 3**

Key parameter settings.

| Algorithm |   | NMA |     |     |     | NNIA |     |     | NSGA-II |     |     | NSGA-III |     |     |
|-----------|---|-----|-----|-----|-----|------|-----|-----|---------|-----|-----|----------|-----|-----|
| Parameter |   | NP  | CP  | MP  | LP  | NP   | CP  | MP  | NP      | CP  | MP  | NP       | CP  | MP  |
| Level     | 1 | 60  | 0.4 | 0.1 | 0.2 | 60   | 0.4 | 0.1 | 60      | 0.4 | 0.1 | 60       | 0.4 | 0.1 |
|           | 2 | 90  | 0.5 | 0.2 | 0.3 | 90   | 0.5 | 0.2 | 90      | 0.5 | 0.2 | 90       | 0.5 | 0.2 |
|           | 3 | 120 | 0.6 | 0.3 | 0.4 | 120  | 0.6 | 0.3 | 120     | 0.6 | 0.3 | 120      | 0.6 | 0.3 |
|           | 4 | 150 | 0.7 | 0.4 | 0.5 | 150  | 0.7 | 0.4 | 150     | 0.7 | 0.4 | 150      | 0.7 | 0.4 |

example,  $[O_{22}, O_{42}]$  is the selected critical block.

**Step 2.** If the priority of job 4 is higher than that of job 2, swap operations on the key blocks to reduce the makespan.

In the example, the makespan of the scheduling scheme is reduced from 64 (shown in Fig. 11(a)) to 54 (shown in Fig. 11(b)) by swapping operations  $O_{22}$  and  $O_{42}$ .

An example of MEM is shown in Fig. 11(c) as follows:

**Step 1.** Find the critical path and the critical blocks.

**Step 2.** Randomly find an operation from the critical blocks, and change its processing machine to another optional machine with minimum cumulative workload. For example, the makespan of the scheduling scheme is reduced from 64 (shown in Fig. 11(a)) to 60 (shown in Fig. 11(c)) by changing the processing machine of  $O_{22}$  from  $M_4$  to  $M_3$  (with minimum cumulative workload) in workshop 11 of factory 1.

**Step 3.** Loop through the above steps until a possible better solution is found or all the critical blocks are traversed.

## 5. Experiments and analysis

The proposed NMA for the DFJSPJP was executed in MATLAB R2020a and run on a computer with an Intel Core i5 2.9 GHz and 16 GB RAM. To test the performance of NMA, three experiments were carried out. Firstly, the key parameters of four algorithms, including NMA, were determined through Taguchi experiments [46,47], and NMA was compared with the other three algorithms. Then, verify the validity of the operators proposed in this paper. Finally, the superior performance of NMA algorithm is proved by comparing with other algorithms. Each benchmark instance was run 10 times to reduce randomness in the experimental results.

### 5.1. Constructing the DFJSPJP benchmarks

In order to evaluate the effectiveness of the proposed NMA in dealing with the DFJSPJP model, a set of 60 DFJSPJP benchmark instances (named DDs 01–60) were created based on established examples of the traditional DFJSPJP standard. These instances are categorized into three sizes: small benchmark instances (DDs 01–20), medium benchmark instances (DDs 21–40), and large benchmark instances (DDs 41–60). In all the instances developed, the machine's power consumption of each processing unit time is randomly set between 5 and 15; the unit transport time between different factories, workshops, and machines are defined as 20, 10, and 5, respectively; the unit energy consumption resulting from unit transportation time is set to 10. The generated benchmark instances DDs 01–60 are available for download from the following website: <https://github.com/Heao1234/Heao>.

### 5.2. Performance evaluation metrics

Three widely used metrics were chosen to test the performance of the proposed algorithm and its operators, including the C-metric [48,49], inverted generational distance (IGD) [50,51] and hypervolume metric (HV) [52]. For the C-metric, the larger the value of C (A, B), the better

**Table 4**

Orthogonal array and results.

| NO. | Parameters |    |    |    | IGD    |        |        |
|-----|------------|----|----|----|--------|--------|--------|
|     | NP         | CP | MP | LP | DD 01  | DD 32  | DD 54  |
| 1   | 1          | 1  | 1  | 1  | 0.2172 | 0.2404 | 0.2227 |
| 2   | 1          | 2  | 2  | 2  | 0.1824 | 0.2120 | 0.2159 |
| 3   | 1          | 3  | 3  | 3  | 0.1740 | 0.1522 | 0.1888 |
| 4   | 1          | 4  | 4  | 4  | 0.1837 | 0.1626 | 0.1896 |
| 5   | 2          | 1  | 2  | 3  | 0.1634 | 0.1975 | 0.1977 |
| 6   | 2          | 2  | 1  | 4  | 0.2068 | 0.1653 | 0.1583 |
| 7   | 2          | 3  | 4  | 1  | 0.1607 | 0.1698 | 0.1793 |
| 8   | 2          | 4  | 3  | 2  | 0.1165 | 0.1945 | 0.1110 |
| 9   | 3          | 1  | 3  | 4  | 0.1517 | 0.1784 | 0.1970 |
| 10  | 3          | 2  | 4  | 3  | 0.1037 | 0.1762 | 0.1719 |
| 11  | 3          | 3  | 1  | 2  | 0.1118 | 0.1631 | 0.1778 |
| 12  | 3          | 4  | 2  | 1  | 0.1622 | 0.1317 | 0.1701 |
| 13  | 4          | 1  | 4  | 2  | 0.1254 | 0.1488 | 0.1464 |
| 14  | 4          | 2  | 3  | 1  | 0.1074 | 0.1700 | 0.1615 |
| 15  | 4          | 3  | 2  | 4  | 0.1234 | 0.1265 | 0.1246 |
| 16  | 4          | 4  | 1  | 3  | 0.1577 | 0.1451 | 0.1430 |

the solution set of A, and vice versa; for the IGD, the lower the IGD value, the better the algorithm performance, and vice versa; and for the HV, the higher the HV value, the better the algorithm performance, and vice versa. More specific descriptions can be seen from the corresponding references.

### 5.3. Parameter settings

Different parameters affect the overall performance of the algorithm, so it is necessary to tune the settings of the key parameters. In this paper, the key parameters include population size (NP), crossover probability (CP), mutation probability (MP), and local search probability (LP). Here, the Taguchi experimental method is employed to calibrate the level of each key parameter in NMA and the three competitive algorithms. Specifically, we designed an orthogonal experiment containing 16 parameter combinations, as shown in columns 2–5 of Table 4. The four levels for each parameter are set as follows: NP = [60, 90, 120, 150], CP = [0.40, 0.50, 0.60, 0.70], MP = [0.10, 0.20, 0.30, 0.40], and LP = [0.20, 0.30, 0.40, 0.50]. Three instances were selected for the experiment including small-scale (DD01), medium-scale (DD 32) and large-scale (DD 54), and the IGD was selected as the evaluation index for the experimental results.

#### 5.3.2. Parameter calibration of the comparison algorithms

In this paper, NNIA [53], NSGA-II [54], and NSGA-III [5,55] were selected as comparison algorithms. The main reasons for choosing these three comparison algorithms are as follows: (1) all of the NNIA, NSGA-II, NSGA-III, and NMA are based on traditional genetic algorithms, so they have similar frameworks; (2) all these algorithms use the non-dominated sorting method to deal with multi-objective problems; and (3) all these algorithms are classical algorithms are usually used as the baseline algorithms to verify the effectiveness of the newly proposed multi-objective algorithms.

To ensure the fairness of the experiment, the Taguchi experiment is also used to calibrate the key parameters of the NNIA, NSGA-II and NSGA-III based on three instances (DD 01, DD 32, and DD 54).

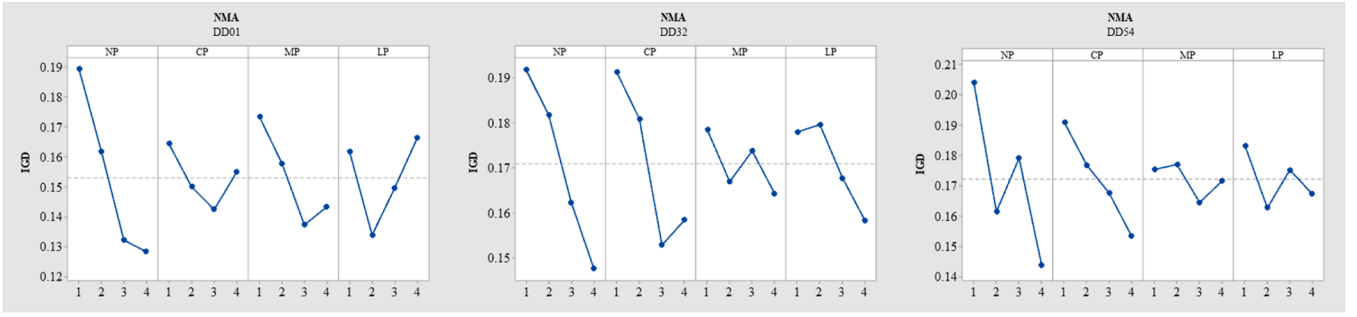


Fig. 12. Main effect of key parameters of NMA.

Table 5

Tuning results of the key parameters of the comparison algorithms.

| Algorithm<br>Parameter | NMA |     |     |     | NNIA |     |     | NSGA-II |     |     | NSGA-III |     |     |
|------------------------|-----|-----|-----|-----|------|-----|-----|---------|-----|-----|----------|-----|-----|
|                        | NP  | CP  | MP  | LP  | NP   | CP  | MP  | NP      | CP  | MP  | NP       | CP  | MP  |
| Level                  | 4   | 3   | 3   | 2   | 3    | 1   | 4   | 3       | 1   | 4   | 4        | 1   | 2   |
| Value                  | 150 | 0.6 | 0.3 | 0.3 | 120  | 0.4 | 0.4 | 120     | 0.4 | 0.4 | 150      | 0.4 | 0.2 |

The key parameters and level settings of each algorithm are shown in Table 3. The variation trend of each parameter level of the three comparison algorithms is plotted in Fig. A1. As shown in Table 5, the key parameter combinations of NNIA, NSGA-II, and NSGA-III were obtained using the same analytical method as NMA.

#### 5.3.1. Parameter calibration of NMA

According to the Taguchi experimental method, an orthogonal table is used to set the parameter combinations of the experiments. In short, in the orthogonal table, the total number of occurrences of each level in each column (representing each factor) is equal; and the level combination of different rows in any two columns is different. Hence, based on the parameter settings in Table 3, we can obtain an orthogonal table with 16 degrees of freedom. As shown in Table 4, a total of 16 parameter combinations are shown in columns 2–5 and the IGD value of parameter combinations are shown in columns 6–8.

Based on the results in Table 4, the rank changes of NMA parameters are shown in Fig. 12. For the NP, the minimum value of IGD is found at the fourth level for DD 01, DD 32, and DD 54; hence, the NP is set as 150. For the CP, the minimum value of IGD is found at the third level for DD 01 and DD 32, and the minimum value of IGD is found at the fourth level for DD 54, hence, the CP is set as 0.6. For the MP, the minimum value of IGD is found at the third level for DD 01 and DD 54, and the minimum value of IGD is found at the fourth level for DD 32, hence, the MP is set as 0.3. For the LP, the minimum value of IGD is found at the second level

for DD 01 and DD 54 and at the fourth level for DD 32, hence, the LP is set as 0.3.

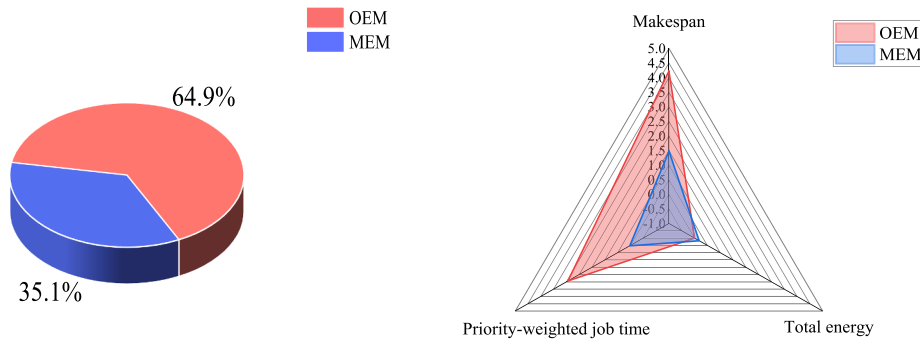
Based on the above result analysis, the optimal key parameters of the NMA are set as follows: NP = 150, CP = 0.6, MP = 0.3, and LP = 0.3 (shown in Table 5).

#### 5.4. Evaluate the effectiveness of the proposed LSO

To validate the efficacy of LSO in the NMA algorithm, we conducted a comparative analysis between NMA with LSO and NMA without LSO (NMA1). A set of 60 instances varying in size (large, medium, and small) were addressed using both NMA versions, and their performance was assessed based on C-metric, IGD, and HV. Each experiment was repeated independently 10 times.

The experimental results of NMA and NMA1 are shown in Table A1 of the Appendix. It can be seen from the experimental results that the performance of NMA is obviously better than that of NMA1. Therefore, it is proved that LSO expands the solution space in the search process, improves the convergence speed of the algorithm, and enables the algorithm to obtain better solutions.

On this basis, we further analyzed the individual impact of the operation exchange method (OEM) and the machine exchange method (MEM) within the local search operation (LSO) on solution quality. Through experiments on 60 instances, we evaluated the usage frequency of OEM and MEM and their impact on the objective functions, as shown



(a) The usage frequencies of OEM and MEM (b) Improvement comparison of OEM and MEM

Fig. 13. Frequency of use of OEM and MEM and comparison of improvements in the three objectives.



**Table 6**

C-metric of the compared algorithms under the stopping standard of the same time.

| Benchmarks         | Size<br>$n \times f \times w \times m$ | D:NMA<br>C(D,D1)  | D1:NNIA<br>C(D1,D) | D:NMA<br>C(D,D2)  | D2:NSGA-II<br>C(D2,D) | D:NMA<br>C(D,D3) | D3:NSGA-III<br>C(D3,D) |
|--------------------|----------------------------------------|-------------------|--------------------|-------------------|-----------------------|------------------|------------------------|
| DD01               | 20×2 × 2 × 3                           | <b>1.0000</b>     | 0.0000             | <b>1.0000</b>     | 0.0000                | <b>1.0000</b>    | 0.0000                 |
| DD02               | 20×2 × 2 × 3                           | <b>1.0000</b>     | 0.0000             | <b>1.0000</b>     | 0.0000                | <b>1.0000</b>    | 0.0000                 |
| DD03               | 20×2 × 2 × 3                           | <b>0.8974</b>     | 0.0625             | <b>0.9318</b>     | 0.0000                | <b>1.0000</b>    | 0.0000                 |
| DD04               | 20×2 × 2 × 3                           | <b>0.6585</b>     | 0.2353             | <b>0.9592</b>     | 0.2353                | <b>1.0000</b>    | 0.0000                 |
| DD05               | 20×2 × 2 × 3                           | <b>1.0000</b>     | 0.0000             | <b>0.7800</b>     | 0.0000                | <b>1.0000</b>    | 0.0000                 |
| DD06               | 10×2 × 2 × 3                           | <b>1.0000</b>     | 0.0000             | <b>1.0000</b>     | 0.0000                | <b>1.0000</b>    | 0.0000                 |
| DD07               | 10×2 × 2 × 3                           | 0.4583            | <b>0.6875</b>      | <b>1.0000</b>     | 0.0625                | <b>1.0000</b>    | 0.0000                 |
| DD08               | 10×2 × 2 × 3                           | <b>1.0000</b>     | 0.0000             | <b>1.0000</b>     | 0.0000                | <b>1.0000</b>    | 0.0000                 |
| DD09               | 10×2 × 2 × 3                           | <b>0.9259</b>     | 0.1250             | <b>1.0000</b>     | 0.1250                | <b>1.0000</b>    | 0.0000                 |
| DD10               | 10×2 × 2 × 3                           | <b>0.6222</b>     | 0.1111             | <b>0.8980</b>     | 0.0000                | <b>1.0000</b>    | 0.0000                 |
| DD11               | 10×2 × 2 × 3                           | 0.2581            | <b>0.6154</b>      | <b>1.0000</b>     | 0.0000                | <b>1.0000</b>    | 0.0000                 |
| DD12               | 10×2 × 2 × 3                           | <b>0.6364</b>     | 0.0000             | <b>0.7333</b>     | 0.1667                | <b>1.0000</b>    | 0.0000                 |
| DD13               | 10×2 × 2 × 3                           | <b>1.0000</b>     | 0.2143             | <b>1.0000</b>     | 0.0000                | <b>1.0000</b>    | 0.0000                 |
| DD14               | 10×2 × 2 × 3                           | 0.6250            | <b>0.6471</b>      | <b>0.8718</b>     | 0.0000                | <b>1.0000</b>    | 0.0000                 |
| DD15               | 10×2 × 2 × 3                           | 0.2222            | <b>0.6250</b>      | <b>1.0000</b>     | 0.1250                | <b>1.0000</b>    | 0.0000                 |
| DD16               | 10×2 × 2 × 3                           | <b>0.9714</b>     | 0.0000             | <b>0.8947</b>     | 0.0000                | <b>1.0000</b>    | 0.0000                 |
| DD17               | 10×2 × 2 × 3                           | <b>1.0000</b>     | 0.0000             | <b>1.0000</b>     | 0.0000                | <b>1.0000</b>    | 0.0000                 |
| DD18               | 10×2 × 2 × 3                           | <b>1.0000</b>     | 0.0000             | <b>1.0000</b>     | 0.0000                | <b>1.0000</b>    | 0.0000                 |
| DD19               | 10×2 × 2 × 3                           | <b>0.5000</b>     | 0.1000             | <b>0.8667</b>     | 0.1000                | <b>1.0000</b>    | 0.0000                 |
| DD20               | 10×2 × 2 × 3                           | <b>0.8409</b>     | 0.0000             | <b>0.9778</b>     | 0.0000                | <b>1.0000</b>    | 0.0000                 |
| DD21               | 20×3 × 3 × 5                           | <b>1.0000</b>     | 0.2667             | <b>0.8065</b>     | 0.2667                | <b>1.0000</b>    | 0.0667                 |
| DD22               | 20×3 × 3 × 5                           | <b>1.0000</b>     | 0.3636             | <b>1.0000</b>     | 0.2727                | <b>1.0000</b>    | 0.0000                 |
| DD23               | 20×3 × 3 × 5                           | <b>1.0000</b>     | 0.0000             | <b>1.0000</b>     | 0.0000                | <b>1.0000</b>    | 0.0000                 |
| DD24               | 15×3 × 3 × 5                           | <b>1.0000</b>     | 0.0000             | <b>1.0000</b>     | 0.0000                | <b>1.0000</b>    | 0.0000                 |
| DD25               | 15×3 × 3 × 5                           | <b>1.0000</b>     | 0.0000             | <b>0.8400</b>     | 0.3571                | <b>1.0000</b>    | 0.0000                 |
| DD26               | 15×3 × 3 × 5                           | <b>0.9844</b>     | 0.2222             | <b>1.0000</b>     | 0.2222                | <b>1.0000</b>    | 0.0000                 |
| DD27               | 15×3 × 3 × 5                           | <b>1.0000</b>     | 0.0000             | <b>1.0000</b>     | 0.0000                | <b>1.0000</b>    | 0.0000                 |
| DD28               | 20×3 × 3 × 5                           | <b>1.0000</b>     | 0.0526             | <b>1.0000</b>     | 0.0000                | <b>1.0000</b>    | 0.0000                 |
| DD29               | 20×3 × 3 × 5                           | <b>1.0000</b>     | 0.0000             | <b>1.0000</b>     | 0.0000                | <b>1.0000</b>    | 0.0000                 |
| DD30               | 20×3 × 3 × 5                           | <b>1.0000</b>     | 0.0000             | <b>1.0000</b>     | 0.0952                | <b>1.0000</b>    | 0.0000                 |
| DD31               | 20×3 × 3 × 5                           | <b>0.9091</b>     | 0.1875             | <b>0.9333</b>     | 0.0000                | <b>1.0000</b>    | 0.0000                 |
| DD32               | 20×3 × 3 × 5                           | <b>1.0000</b>     | 0.0000             | <b>1.0000</b>     | 0.0625                | <b>1.0000</b>    | 0.0000                 |
| DD33               | 15×3 × 3 × 5                           | <b>1.0000</b>     | 0.0000             | <b>1.0000</b>     | 0.0000                | <b>1.0000</b>    | 0.0000                 |
| DD34               | 15×3 × 3 × 5                           | <b>1.0000</b>     | 0.0000             | <b>1.0000</b>     | 0.0000                | <b>1.0000</b>    | 0.0000                 |
| DD35               | 15×3 × 3 × 5                           | <b>1.0000</b>     | 0.0000             | 0.0000            | <b>1.0000</b>         | <b>1.0000</b>    | 0.0000                 |
| DD36               | 15×3 × 3 × 5                           | <b>1.0000</b>     | 0.0000             | <b>1.0000</b>     | 0.0000                | <b>1.0000</b>    | 0.0000                 |
| DD37               | 15×3 × 3 × 5                           | <b>0.9583</b>     | 0.0000             | <b>1.0000</b>     | 0.0000                | <b>1.0000</b>    | 0.0000                 |
| DD38               | 15×3 × 3 × 5                           | <b>1.0000</b>     | 0.0000             | <b>1.0000</b>     | 0.0000                | <b>1.0000</b>    | 0.0000                 |
| DD39               | 15×3 × 3 × 5                           | <b>1.0000</b>     | 0.0000             | <b>1.0000</b>     | 0.0000                | <b>1.0000</b>    | 0.0000                 |
| DD40               | 15×4 × 4 × 7                           | <b>1.0000</b>     | 0.1538             | <b>1.0000</b>     | 0.0000                | <b>1.0000</b>    | 0.0000                 |
| DD41               | 15×4 × 4 × 7                           | <b>1.0000</b>     | 0.0000             | <b>1.0000</b>     | 0.1538                | <b>1.0000</b>    | 0.0000                 |
| DD42               | 20×4 × 4 × 7                           | <b>1.0000</b>     | 0.0400             | <b>1.0000</b>     | 0.0400                | <b>1.0000</b>    | 0.0000                 |
| DD43               | 20×4 × 4 × 7                           | <b>1.0000</b>     | 0.0000             | <b>1.0000</b>     | 0.0000                | <b>1.0000</b>    | 0.0000                 |
| DD44               | 20×4 × 4 × 7                           | <b>1.0000</b>     | 0.0000             | <b>1.0000</b>     | 0.0000                | <b>1.0000</b>    | 0.0000                 |
| DD45               | 20×4 × 4 × 7                           | <b>1.0000</b>     | 0.1429             | <b>0.9804</b>     | 0.1429                | <b>1.0000</b>    | 0.0000                 |
| DD46               | 20×4 × 4 × 7                           | <b>1.0000</b>     | 0.0000             | <b>1.0000</b>     | 0.0000                | <b>1.0000</b>    | 0.0000                 |
| DD47               | 20×4 × 4 × 7                           | <b>1.0000</b>     | 0.0000             | <b>1.0000</b>     | 0.0000                | <b>1.0000</b>    | 0.0000                 |
| DD48               | 30×4 × 4 × 7                           | <b>1.0000</b>     | 0.0000             | <b>1.0000</b>     | 0.0000                | <b>1.0000</b>    | 0.0000                 |
| DD49               | 30×4 × 4 × 7                           | <b>0.8667</b>     | 0.1765             | <b>1.0000</b>     | 0.0588                | <b>1.0000</b>    | 0.0000                 |
| DD50               | 30×4 × 4 × 7                           | <b>1.0000</b>     | 0.0000             | 0.0000            | <b>0.2500</b>         | <b>1.0000</b>    | 0.0000                 |
| DD51               | 30×4 × 4 × 7                           | <b>1.0000</b>     | 0.0000             | <b>1.0000</b>     | 0.0000                | <b>1.0000</b>    | 0.0000                 |
| DD52               | 30×4 × 4 × 7                           | <b>1.0000</b>     | 0.0556             | <b>1.0000</b>     | 0.0000                | <b>1.0000</b>    | 0.0000                 |
| DD53               | 20×4 × 4 × 7                           | <b>1.0000</b>     | 0.0000             | <b>1.0000</b>     | 0.0000                | <b>1.0000</b>    | 0.0000                 |
| DD54               | 20×4 × 4 × 7                           | <b>1.0000</b>     | 0.0000             | <b>1.0000</b>     | 0.0000                | <b>1.0000</b>    | 0.0000                 |
| DD55               | 20×4 × 4 × 7                           | <b>1.0000</b>     | 0.0000             | <b>1.0000</b>     | 0.1667                | <b>1.0000</b>    | 0.0000                 |
| DD56               | 30×4 × 4 × 7                           | <b>1.0000</b>     | 0.0000             | <b>1.0000</b>     | 0.0000                | <b>1.0000</b>    | 0.0000                 |
| DD57               | 30×4 × 4 × 7                           | <b>1.0000</b>     | 0.0000             | <b>1.0000</b>     | 0.0000                | <b>1.0000</b>    | 0.0000                 |
| DD58               | 30×4 × 4 × 7                           | <b>1.0000</b>     | 0.0000             | <b>1.0000</b>     | 0.0000                | <b>1.0000</b>    | 0.0000                 |
| DD59               | 30×4 × 4 × 7                           | <b>0.8636</b>     | 0.0500             | <b>1.0000</b>     | 0.0500                | <b>1.0000</b>    | 0.0000                 |
| DD60               | 30×4 × 4 × 7                           | <b>1.0000</b>     | 0.0000             | <b>1.0000</b>     | 0.0000                | <b>1.0000</b>    | 0.0000                 |
| Win numbers (Rate) |                                        | <b>56(93.3 %)</b> | <b>4(6.7 %)</b>    | <b>58(96.7 %)</b> | <b>2(3.3 %)</b>       | <b>60(100 %)</b> | <b>0(0 %)</b>          |

in Fig. 13. The experimental results show that the usage frequencies of OEM and MEM are 64.89 % and 35.11 % respectively, indicating that OEM is more frequently invoked to improve solutions during the optimization process. For the three objective functions, OEM demonstrates significant improvement effects in makespan (4.22 %) and weighted priority job time (2.94 %). However, in terms of total energy consumption, OEM shows almost no improvement, while MEM exhibits a stronger improvement capability (0.18 %). In conclusion, OEM performs more prominently in optimizing makespan and weighted average time of job priorities, while MEM compensates for OEM's shortcomings in

total energy consumption optimization. This complementarity suggests that combining the strengths of OEM and MEM can further enhance the overall performance of the algorithm.

##### 5.5. Comparison with other algorithms

To further confirm the effectiveness of the proposed NMA algorithm, it was compared with three well-known multi-objective algorithms [56, 57]: NNIA, NSGA-II, and NSGA-III. To ensure a fair comparison, NNIA, NSGA-II, and NSGA-III employed identical initialization methods,

**Table 7**

HV and IGD value of the compared algorithms under the stopping standard of the same time.

| Benchmarks         | HV             |               |            |            | IGD           |               |               |            |
|--------------------|----------------|---------------|------------|------------|---------------|---------------|---------------|------------|
|                    | NMA            | NNIA          | NSGA-II    | NSGA-III   | NMA           | NNIA          | NSGA-II       | NSGA-III   |
| DD01               | <b>0.0300</b>  | 0.0135        | 0.0174     | 0.0032     | 0.4439        | 0.2939        | <b>0.2185</b> | 0.6796     |
| DD02               | <b>0.0316</b>  | 0.0211        | 0.0158     | 0.0024     | 0.3034        | <b>0.1536</b> | 0.2330        | 0.7496     |
| DD03               | <b>0.0249</b>  | 0.0158        | 0.0119     | 0.0006     | <b>0.0661</b> | 0.1838        | 0.2773        | 0.9042     |
| DD04               | <b>0.0148</b>  | 0.0115        | 0.0100     | 0.0008     | 0.1843        | <b>0.1607</b> | 0.1817        | 0.9158     |
| DD05               | <b>0.0279</b>  | 0.0191        | 0.0238     | 0.0055     | 0.1655        | 0.2005        | <b>0.1425</b> | 0.5905     |
| DD06               | <b>0.0243</b>  | 0.0158        | 0.0126     | 0.0024     | <b>0.0185</b> | 0.3109        | 0.3401        | 0.8521     |
| DD07               | <b>0.0278</b>  | 0.0240        | 0.0203     | 0.0038     | 0.1053        | <b>0.0316</b> | 0.1630        | 0.7741     |
| DD08               | <b>0.0427</b>  | 0.0322        | 0.0288     | 0.0065     | <b>0.0000</b> | 0.2167        | 0.2839        | 0.7980     |
| DD09               | <b>0.0282</b>  | 0.0196        | 0.0171     | 0.0020     | <b>0.1162</b> | 0.1572        | 0.2273        | 0.7879     |
| DD10               | <b>0.0317</b>  | 0.0269        | 0.0218     | 0.0046     | 0.2851        | <b>0.0405</b> | 0.1542        | 0.6184     |
| DD11               | 0.0226         | <b>0.0291</b> | 0.0190     | 0.0019     | 0.0639        | <b>0.0301</b> | 0.1395        | 0.7727     |
| DD12               | <b>0.0211</b>  | 0.0184        | 0.0138     | 0.0006     | <b>0.0586</b> | 0.0781        | 0.1529        | 0.8096     |
| DD13               | <b>0.0429</b>  | 0.0283        | 0.0227     | 0.0073     | <b>0.0942</b> | 0.1928        | 0.3350        | 0.7030     |
| DD14               | <b>0.0246</b>  | 0.0192        | 0.0134     | 0.0042     | 0.1802        | <b>0.0774</b> | 0.2050        | 0.5211     |
| DD15               | <b>0.0376</b>  | 0.0319        | 0.0172     | 0.0024     | 0.0728        | <b>0.0406</b> | 0.2882        | 0.8075     |
| DD16               | <b>0.0227</b>  | 0.0141        | 0.0144     | 0.0030     | <b>0.1318</b> | 0.1471        | 0.1903        | 0.6834     |
| DD17               | <b>0.0348</b>  | 0.0184        | 0.0231     | 0.0015     | <b>0.0000</b> | 0.4491        | 0.2822        | 1.0658     |
| DD18               | <b>0.0352</b>  | 0.0144        | 0.0149     | 0.0015     | <b>0.0000</b> | 0.5536        | 0.4375        | 1.0089     |
| DD19               | <b>0.0214</b>  | 0.0125        | 0.0107     | 0.0007     | 0.3819        | <b>0.1791</b> | 0.2673        | 0.9043     |
| DD20               | <b>0.0269</b>  | 0.0172        | 0.0153     | 0.0022     | <b>0.0724</b> | 0.2525        | 0.3203        | 0.7846     |
| DD21               | <b>0.0176</b>  | 0.0135        | 0.0148     | 0.0064     | <b>0.0935</b> | 0.1479        | 0.1178        | 0.4367     |
| DD22               | <b>0.0128</b>  | 0.0096        | 0.0099     | 0.0038     | <b>0.1000</b> | 0.1601        | 0.1197        | 0.4962     |
| DD23               | <b>0.0257</b>  | 0.0138        | 0.0113     | 0.0029     | <b>0.0000</b> | 0.3397        | 0.4503        | 0.8202     |
| DD24               | <b>0.0194</b>  | 0.0074        | 0.0077     | 0.0031     | <b>0.0000</b> | 0.3645        | 0.3311        | 0.5767     |
| DD25               | <b>0.0122</b>  | 0.0054        | 0.0064     | 0.0020     | 0.2597        | 0.3055        | <b>0.2192</b> | 0.6217     |
| DD26               | <b>0.0216</b>  | 0.0121        | 0.0109     | 0.0030     | <b>0.1881</b> | 0.2715        | 0.3254        | 0.6861     |
| DD27               | <b>0.0141</b>  | 0.0104        | 0.0105     | 0.0040     | <b>0.1230</b> | 0.1375        | 0.1295        | 0.4827     |
| DD28               | <b>0.0169</b>  | 0.0081        | 0.0084     | 0.0019     | <b>0.0000</b> | 0.3803        | 0.3649        | 0.8633     |
| DD29               | <b>0.0185</b>  | 0.0073        | 0.0074     | 0.0011     | <b>0.0000</b> | 0.5260        | 0.5075        | 0.9944     |
| DD30               | <b>0.0140</b>  | 0.0099        | 0.0085     | 0.0025     | <b>0.0837</b> | 0.1465        | 0.2452        | 0.6493     |
| DD31               | <b>0.0092</b>  | 0.0070        | 0.0062     | 0.0015     | <b>0.0291</b> | 0.1106        | 0.1624        | 0.6159     |
| DD32               | <b>0.0352</b>  | 0.0195        | 0.0194     | 0.0058     | <b>0.0000</b> | 0.4130        | 0.4197        | 0.7877     |
| DD33               | <b>0.0227</b>  | 0.0150        | 0.0121     | 0.0032     | <b>0.0000</b> | 0.2684        | 0.4096        | 0.9153     |
| DD34               | <b>0.0171</b>  | 0.0074        | 0.0066     | 0.0016     | <b>0.0000</b> | 0.5484        | 0.5718        | 0.9603     |
| DD35               | <b>0.0171</b>  | 0.0062        | 0.0065     | 0.0008     | <b>0.0000</b> | 0.6850        | 0.6315        | 1.0966     |
| DD36               | <b>0.0209</b>  | 0.0105        | 0.0112     | 0.0038     | <b>0.0000</b> | 0.4662        | 0.4255        | 0.7265     |
| DD37               | <b>0.0105</b>  | 0.0089        | 0.0052     | 0.0008     | 0.0753        | <b>0.0663</b> | 0.3684        | 0.8959     |
| DD38               | <b>0.0160</b>  | 0.0079        | 0.0068     | 0.0015     | <b>0.0000</b> | 0.4299        | 0.5349        | 0.9023     |
| DD39               | 0.0078         | <b>0.0172</b> | 0.0059     | 0.0010     | <b>0.0000</b> | 0.5291        | 0.6404        | 1.0555     |
| DD40               | <b>0.0166</b>  | 0.0100        | 0.0067     | 0.0011     | <b>0.0000</b> | 0.3168        | 0.5028        | 0.9554     |
| DD41               | <b>0.0086</b>  | 0.0048        | 0.0045     | 0.0012     | <b>0.0000</b> | 0.3347        | 0.3798        | 0.7734     |
| DD42               | <b>0.0079</b>  | 0.0039        | 0.0035     | 0.0015     | <b>0.0000</b> | 0.4373        | 0.4527        | 0.6607     |
| DD43               | <b>0.0137</b>  | 0.0090        | 0.0090     | 0.0040     | <b>0.0000</b> | 0.2694        | 0.2115        | 0.6103     |
| DD44               | <b>0.0141</b>  | 0.0079        | 0.0081     | 0.0035     | <b>0.0000</b> | 0.3840        | 0.3787        | 0.6185     |
| DD45               | <b>0.0078</b>  | 0.0048        | 0.0045     | 0.0017     | <b>0.1212</b> | 0.1593        | 0.1255        | 0.5328     |
| DD46               | <b>0.0076</b>  | 0.0039        | 0.0039     | 0.0009     | <b>0.0000</b> | 0.3843        | 0.3655        | 0.8674     |
| DD47               | <b>0.0121</b>  | 0.0058        | 0.0067     | 0.0023     | <b>0.0000</b> | 0.5065        | 0.3966        | 0.8594     |
| DD48               | <b>0.0153</b>  | 0.0051        | 0.0064     | 0.0020     | <b>0.1059</b> | 0.4845        | 0.3818        | 0.7054     |
| DD49               | <b>0.0148</b>  | 0.0110        | 0.0084     | 0.0031     | <b>0.1213</b> | 0.1377        | 0.2096        | 0.6050     |
| DD50               | <b>0.0112</b>  | 0.0076        | 0.0081     | 0.0026     | <b>0.0000</b> | 0.2844        | 0.2129        | 0.6352     |
| DD51               | <b>0.0131</b>  | 0.0041        | 0.0056     | 0.0017     | <b>0.0000</b> | 0.5955        | 0.3887        | 0.7592     |
| DD52               | <b>0.0146</b>  | 0.0066        | 0.0063     | 0.0015     | <b>0.0000</b> | 0.4232        | 0.4771        | 0.8728     |
| DD53               | <b>0.0089</b>  | 0.0030        | 0.0036     | 0.0011     | <b>0.0000</b> | 0.6310        | 0.5438        | 0.9724     |
| DD54               | <b>0.0060</b>  | 0.0031        | 0.0039     | 0.0017     | <b>0.0737</b> | 0.2150        | 0.2953        | 0.5581     |
| DD55               | <b>0.0107</b>  | 0.0045        | 0.0054     | 0.0021     | 0.2896        | 0.2038        | <b>0.1833</b> | 0.5035     |
| DD56               | <b>0.0153</b>  | 0.0066        | 0.0057     | 0.0018     | <b>0.0000</b> | 0.4174        | 0.5137        | 0.8566     |
| DD57               | <b>0.0114</b>  | 0.0064        | 0.0071     | 0.0021     | <b>0.0223</b> | 0.3004        | 0.1852        | 0.5962     |
| DD58               | <b>0.0141</b>  | 0.0056        | 0.0063     | 0.0024     | <b>0.0000</b> | 0.5190        | 0.4774        | 0.8170     |
| DD59               | <b>0.0142</b>  | 0.0082        | 0.0080     | 0.0037     | <b>0.0417</b> | 0.1291        | 0.1788        | 0.4873     |
| DD60               | <b>0.0158</b>  | 0.0064        | 0.0077     | 0.0025     | <b>0.0000</b> | 0.4637        | 0.3861        | 0.7051     |
| Win numbers (Rate) | 58<br>(96.7 %) | 2<br>(3.3 %)  | 0<br>(0 %) | 0<br>(0 %) | 47<br>(78 %)  | 9<br>(15 %)   | 4<br>(7 %)    | 0<br>(0 %) |

crossover, and mutation operators as our NMA on 60 benchmark instances, and their key parameters were set according to the calibration results obtained in [Section 5.3](#).

To ensure fairness and reasonableness of the test, the experiment was divided into two parts. Initially, the first part set 100 iterations as the stopping criterion, with each algorithm independently running 10 times. The second part considered the minimum average running time from the first stage for each instance, and each algorithm ran independently 10 times.

In the first part, comparison results based on C-metric, HV, and IGD were documented in [Tables A2 and A3](#) of the Appendix. As evidenced in [Table A2](#), the proposed NMA exhibited superior performance compared to NNIA and NSGA-III in most instances, and outperformed NSGA-II in all instances based on the C-metric. Specifically, 95 %, 100 %, and 100 % of the instances solved by NMA outperformed NNIA, NSGA-II, and NSGA-III, respectively. Similarly, based on HV in [Table A3](#), 98 % of the instances solved by NMA surpassed NNIA, NSGA-II, and NSGA-III, and 76 % of the instances outperformed these algorithms based on IGD.

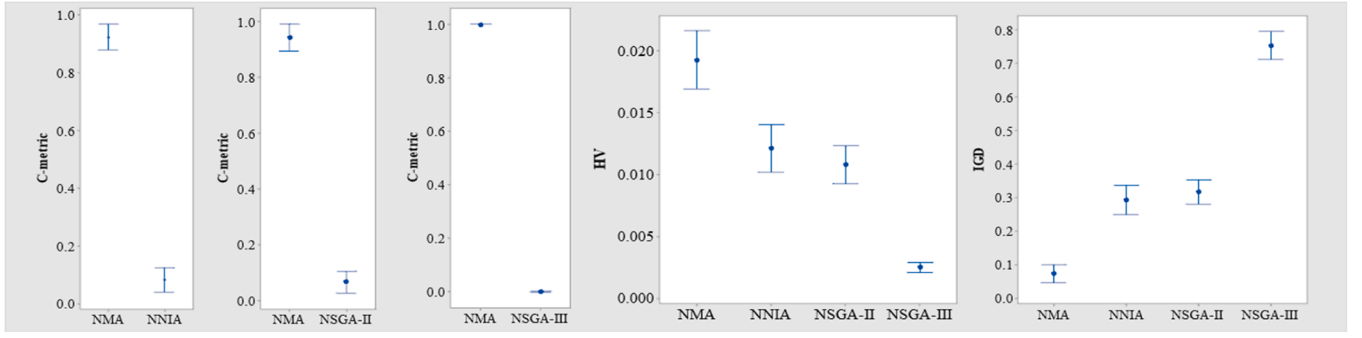


Fig. 14. Mean plots with 95 % CI in terms of C-metric, HV and IGD results of four algorithms.

Table 8

Wilcoxon signed-rank test.

| value           | (C-metric) |         | (HV)     |         | (IGD)    |         |
|-----------------|------------|---------|----------|---------|----------|---------|
|                 | p-value    | h-value | p-value  | h-value | p-value  | h-value |
| (NMA, NNIA)     | 8.98E-12   | 1       | 1.02E-11 | 1       | 8.70E-08 | 1       |
| (NMA, NSGA-II)  | 5.02E-12   | 1       | 1.63E-11 | 1       | 2.37E-09 | 1       |
| (NMA, NSGA-III) | 1.53E-14   | 1       | 1.58E-11 | 1       | 1.63E-11 | 1       |

To ensure fairness and reasonableness of the test, the second part utilized the minimum mean run time as the stopping criterion, as detailed in Table A4 of the Appendix. Each of the four comparison algorithms was independently run 10 times for each instance. From Table 6, it can be seen that NMA outperformed NNIA, NSGA-II, and NSGA-III in 93 %, 96 %, and 100 % of instances based on the C-metric, respectively. Similarly, based on HV in Table 7, NMA outperformed NNIA, NSGA-II, and NSGA-III in 96 % of instances, and outperformed these algorithms in 78 % of instances based on IGD. The experimental results from the two stages indicate that our proposed NMA significantly outperforms NNIA, NSGA-II, and NSGA-III in addressing DFJSPJP problems, regardless of whether the stopping criterion is a fixed number of iterations or a consistent running time.

To check the significant differences between NMA and other three algorithms (NNIA, NSGA-II, and NSGA-III), Fig. 14 shows the mean plots of C-metric, HV, and IGD for the comparison algorithms with 95 % confidence intervals (CI). From Fig. 14, the C-metric interval range of NMA is closer to 1 than that of NNIA, NSGA-III and NSGA-II, indicating that the results of NMA have a higher probability of advantage. The worst values of the HV and IGD interval range of NMA are around 0.018 and 0.08, respectively, which are obviously better than other algorithms. Based on the above experimental results and analysis, the comprehensive performance ranking of the comparison algorithm is: NMA > NNIA > NSGA-II > NSGA-III. On the other hand, a nonparametric statistical test of Wilcoxon signed rank test [58,59] was performed at 95 % confidence level (as shown in Table 8). As can be seen from Table 8, the p-values of the evaluation metrics such as C-metric, HV and IGD are significantly below the 0.05 threshold. Meanwhile, all the corresponding h-values are equal to 1. This indicates that there is statistical significance between the results obtained by the NMA and the other three comparison algorithms [60]. Therefore, it is proved that the NMA algorithm outperforms the other three algorithms from the perspective of statistics.

In order to display the distribution of the non-dominated solutions of the algorithms intuitively, their Pareto front (PF) distribution are depicted in Fig. 15 based on 6 randomly selected instances (DD 02, DC 05, DC 23, DC 25, DC 44, DC 46). From Fig. 15(a)-(f), it can be seen that the solution set of NMA is closer to the bottom of the graph than that of

NNIA, NSGA-II and NSGA-III, which indicates that the solution obtained by NMA has a higher probability to dominate the solutions of other three algorithms.

In order to test the convergence performance of NMA, three instances (DD 02, DD 23, and DD 44) were randomly selected and their average convergence curves were drawn in Fig. 16. As can be seen from the Fig. 16, the objectives of  $f_1$ ,  $f_2$  and  $f_3$  of NMA decreased rapidly in the early iteration, and can obtain the best solutions finally, which demonstrates that the NMA has high performance of convergence. Therefore, it can be concluded that the proposed NMA has better convergence performance than the other three algorithms in solving the DFJSPJP.

Figs. 17 and 18 present the Gantt charts for two different solutions obtained using NMA and NSGA-II for instance DD 03, where jobs 3, 8, and 13 are the jobs with highest priority. As can be seen from the Figs. 17 and 18, the maximum completion times for these priority jobs by NMA and NSGA-II are 498 and 584 for job 3, 448 and 750 for job 13, and 407 and 562 for job 8, respectively. Meanwhile, the makespan of NMA and NSGA-II are nearly identical, at 761 and 768, respectively. This indicates that, with similar makespan, the proposed NMA algorithm can ensure that high-priority jobs are completed earlier than by the NSGA-II algorithm.

To comprehensively evaluate the stability and reliability of the proposed NMA algorithm, we conducted 10 independent runs for each algorithm (NMA, NNIA, NSGA-II, and NSGA-III) on a total of 60 benchmark instances of small, medium, and large sizes. Each run utilized a different random seed, with 100 iterations set as the stopping criterion. To ensure fairness, NNIA, NSGA-II, and NSGA-III employed the same initialization method, crossover, and mutation operators as NMA. From Fig. 19, it is evident that NMA achieves significantly better average values than the three comparison algorithms (NNIA, NSGA-II, and NSGA-III) on the C-metric, HV, and IGD metrics. Furthermore, NMA exhibits significant stability in terms of both the C-metric and IGD indicators, as evidenced by its consistently narrower confidence intervals compared to the three comparison algorithms. This result indicates that NMA demonstrates smaller performance variations across different random seeds, highlighting its superior stability. Although the confidence interval of NMA on the HV indicator is slightly wider than those of the three comparison algorithms, the absence of overlap in their confidence intervals also demonstrates that NMA stably outperforms the three comparison algorithms.

## 5.6. Time complexity analysis

The computational complexity of our proposed NMA (new memetic algorithm) is primarily influenced by four key components: objective function evaluation, fast non-dominated sorting, crowding distance calculation, and the local search operation. For a population of size  $N$  and  $M$  objective functions, the complexity of assessing the objective functions is  $O(MN)$ . The most computationally demanding steps are the fast non-dominated sorting and the crowding distance calculation. Both steps involve sorting individuals for each objective function, resulting in

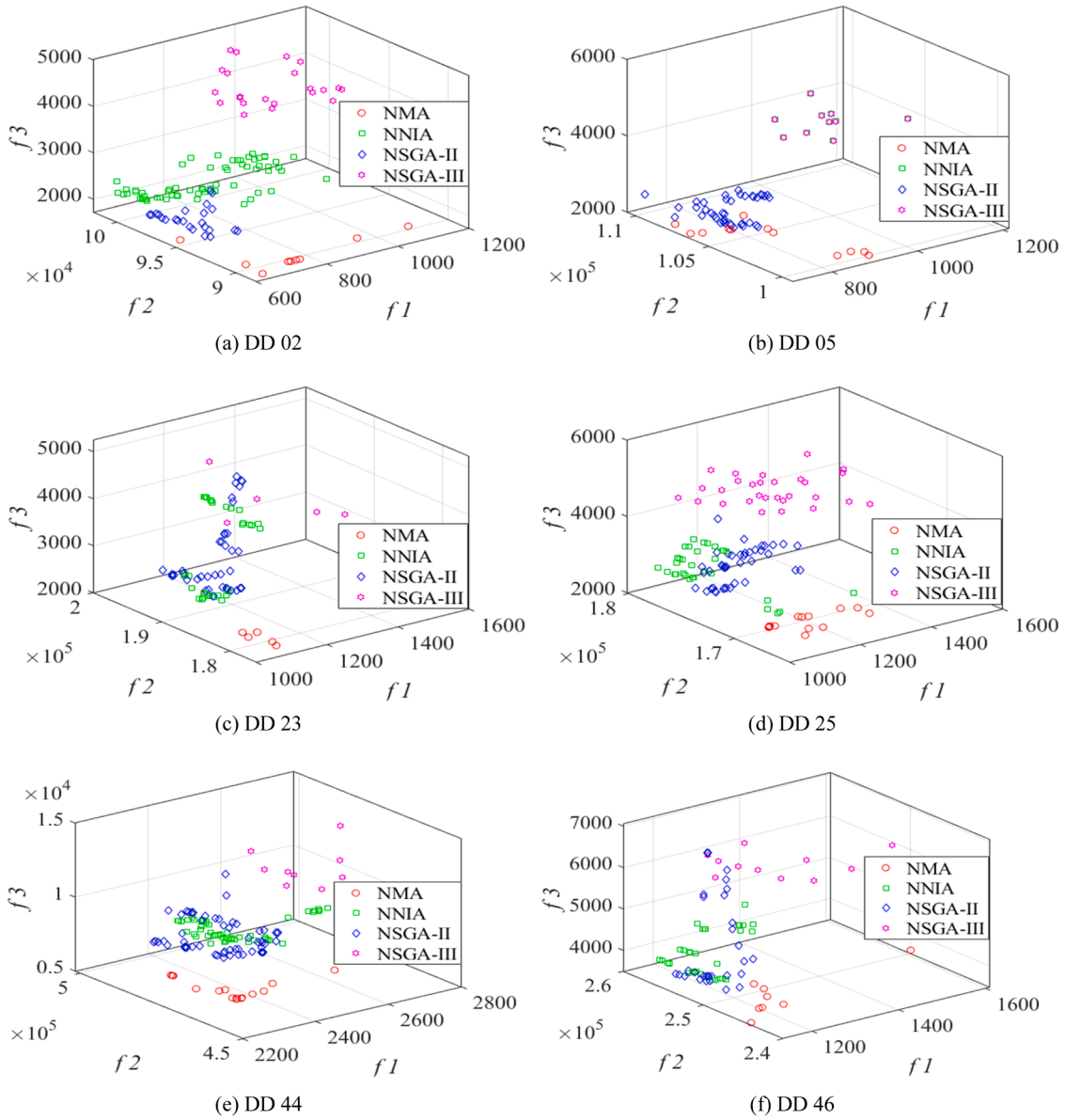


Fig. 15. PF results of the selected instances.

a complexity of  $O(MN^2)$ . The complexity of local search operations is closely related to the complexity of operational exchange method (OEM) and machine exchange method (MEM). The OEM has a complexity of  $O(K \cdot T1 \cdot T2)$ , where  $K$  represents the number of individuals selected from the first level of the population through fast non-dominated sorting, and  $T1$  and  $T2$  are random integers selected from 1 to  $N_{cp}$  (the number of critical blocks on the critical path). The MEM, on the other hand, has a complexity of  $O(K \cdot L1)$ , where  $L1$  is a random integer selected from 1 to  $N_{cp}$ . Combining these components, the overall computational complexity of the NMA is  $O(\max\{MN^2, K \cdot T1 \cdot T2, K \cdot L1\})$ .

For the comparison algorithms NNIA, NSGA-II, and NSGA-III in this paper, their computational complexity is primarily determined by non-dominated sorting and crowding distance calculation (or reference point association), which can be denoted by  $O(MN^2)$ . According to the basic principle of Big-O complexity analysis, the computational complexity of an algorithm is determined by the part with the highest growth rate [61]. And for NMA, NNIA, NSGA-II and NSGA-III, the fastest growing part of the computational complexity is  $O(MN^2)$ . Therefore, they have the equal computational complexity.

### 5.7. The result analysis

In this section, we analyze in detail the performance of the NMA algorithm in 60 test instances. The experimental results show that NMA outperforms NNIA, NSGA-II, and NSGA-III in most instances. Specifically, NMA outperforms NNIA, NSGA-II, and NSGA-III in 95 %, 100 %, and 80 % of instances in C metrics, respectively, which shows a significant advantage in terms of the quality of the solution. In addition, NMA also outperforms in the HV and IGD metrics: in 96 % of the instances, NMA obtains higher HV values, suggesting that its solution set covers a wider range of Pareto frontiers; in 78 % of the instances, NMA obtains lower IGD values, suggesting that its solution set is closer to the true Pareto frontiers. The random seed experiments further demonstrate that NMA achieves narrower confidence intervals across multiple runs, confirming its superior stability and robustness. These results fully demonstrate the strong capability of NMA in multi-objective optimization problems, particularly in terms of solution quality and diversity. From the convergence curves in Fig. 16, it can be observed that NMA converges rapidly in the early iterations and is able to find better solutions in later stages. This phenomenon indicates that, compared to the

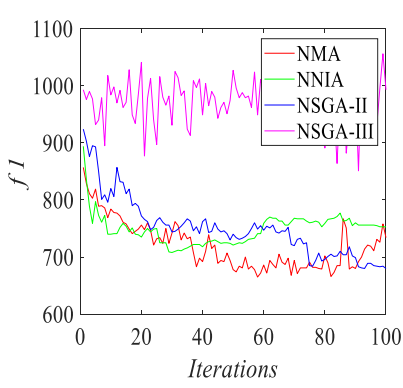
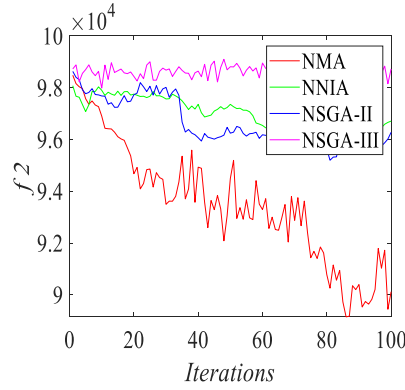
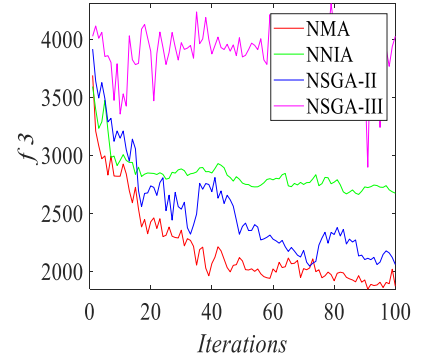
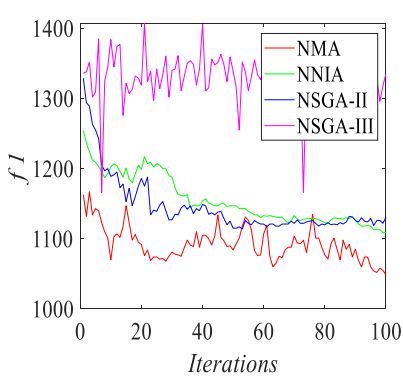
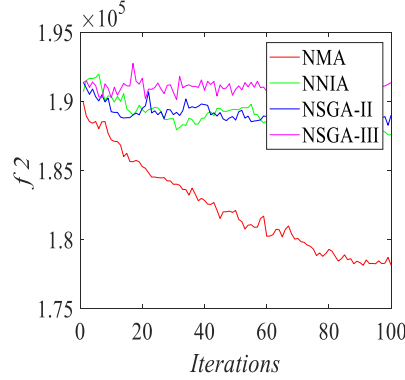
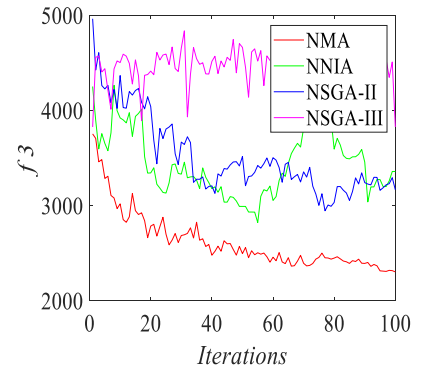
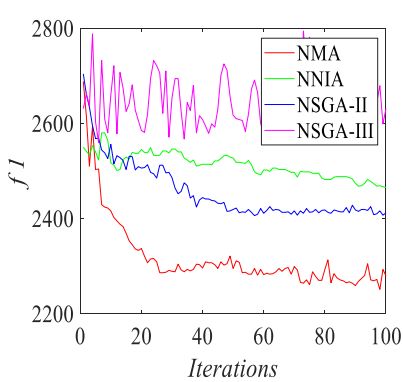
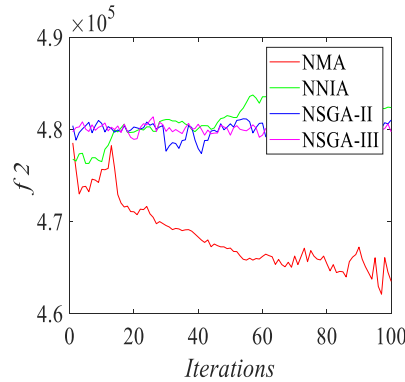
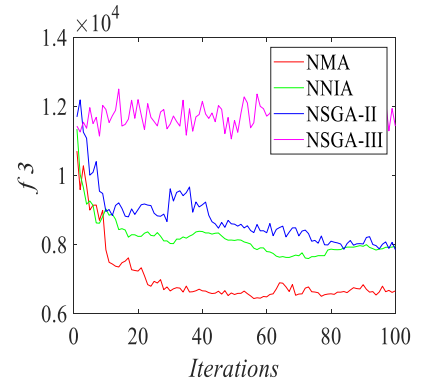
(a) Convergence curves of  $f_1$  for DD 02(b) Convergence curves of  $f_2$  for DD 02(c) Convergence curves of  $f_3$  for DD 02(d) Convergence curves of  $f_1$  for DD 23(e) Convergence curves of  $f_2$  for DD 23(f) Convergence curves of  $f_3$  for DD 23(g) Convergence curves of  $f_1$  for DD 44(h) Convergence curves of  $f_2$  for DD 44(i) Convergence curves of  $f_3$  for DD 44

Fig. 16. Convergence diagrams of DD 02, DD 23 and DD 44.

other three algorithms, NMA not only possesses stronger global search capabilities but also achieves faster convergence speed.

In the analysis of specific instances, we selected DD03, DD23, and DD44 for detailed discussion. From the Gantt charts in Figs. 17 and 18, it can be seen that NMA is able to prioritize high-priority jobs while ensuring the minimization of the makespan. For example, in the DD03 instance, NMA significantly advanced the completion times of high-priority jobs, whereas NSGA-II performed poorly in handling the same jobs. Additionally, the Pareto front distribution in Fig. 15 shows that NMA's solutions are closer to the true Pareto front, further validating its

superiority in multi-objective optimization.

To verify the effectiveness of LSO in the NMA algorithm, we compared the NMA with LSO and the NMA without LSO (NMA1). We tested both versions on 60 instances of different sizes. The results showed that NMA performed significantly better than NMA1. This proves that LSO expands the solution space during the search process, improves the algorithm's convergence speed, and enhances solution quality. Further analysis revealed that OEM excels in optimizing makespan and weighted job priority time. On the other hand, MEM shows stronger performance in optimizing total energy consumption.

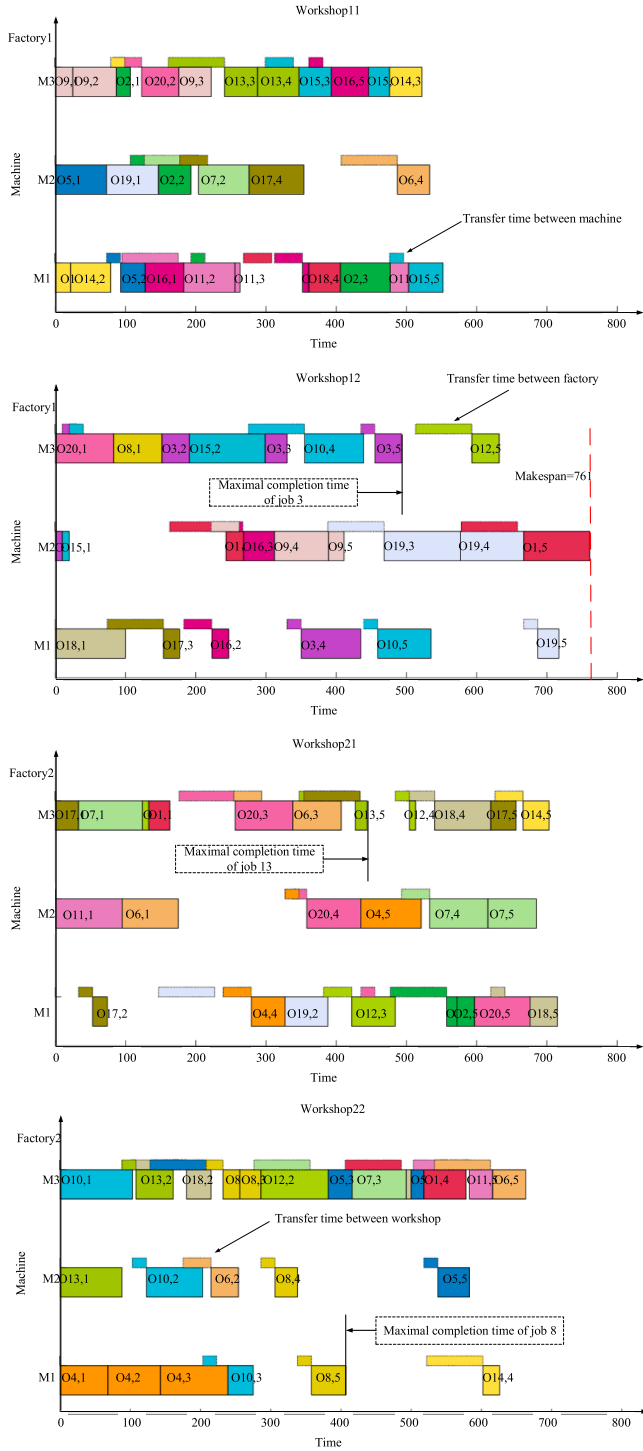


Fig. 17. Gantt chart of a solution of DD 03 with NMA.

The combination of OEM and MEM can further improve the overall performance of the algorithm.

In summary, the NMA proposed in our study offers several advantages. Firstly, it integrates a well-designed chromosome encoding method (CEM) and local search operators (LSO), significantly enhancing the algorithm's ability to generate high-quality initial populations and improving convergence speed. Secondly, experimental results on 60 benchmark instances demonstrate the superiority of NMA in terms of solution quality and computational efficiency compared to other well-known algorithms such as NNIA, NSGA-II, and NSGA-III. Lastly, owing

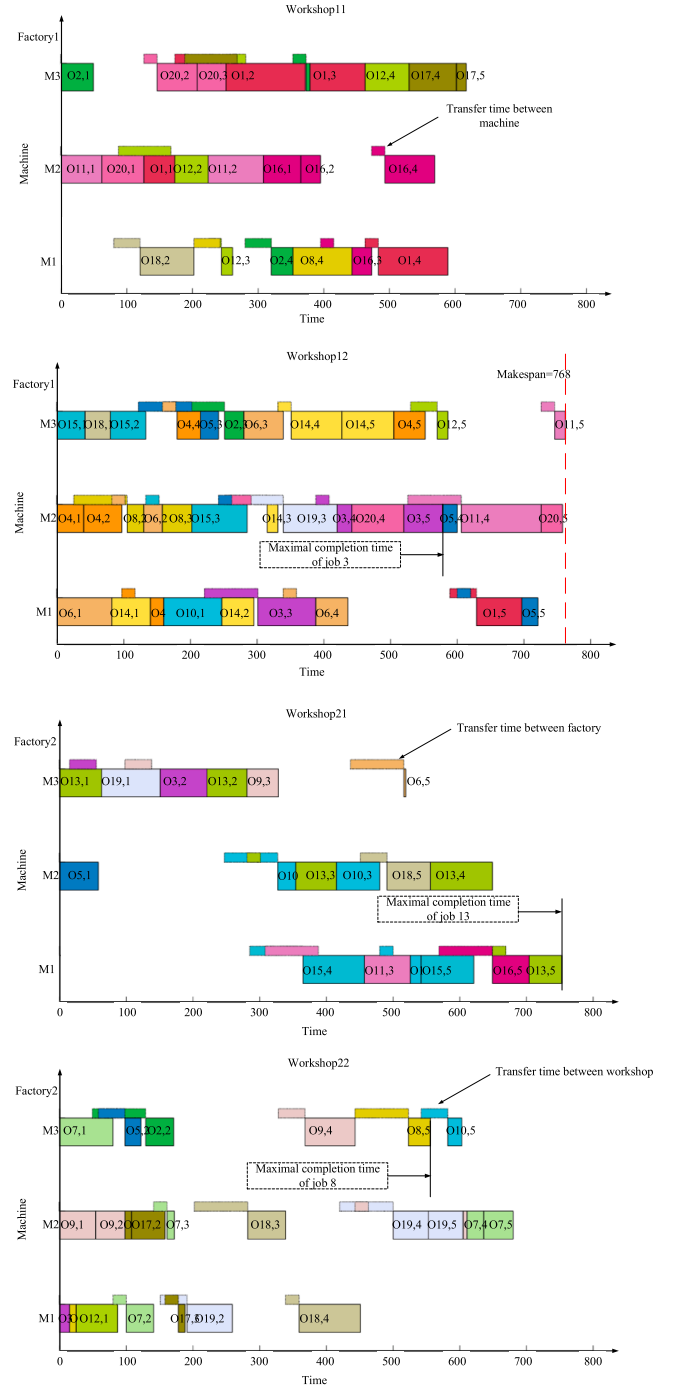


Fig. 18. Gantt chart of a solution of DD 03 with NSGA-II.

to NMA's unique multi-layer encoding mechanism and multi-objective collaborative optimization strategy, it has also shown versatility in addressing multi-objective optimization problems in other fields. Despite its excellent performance in most scenarios, NMA does have some limitations. Its performance is constrained by initial parameters such as population size and crossover probability. For instance, when dealing with small-scale instances, its multi-layer local search mechanism may lead to low efficiency due to redundant computations.

## 6. Conclusion and future research

In this paper, we studied the DFJSPJP considering the completion



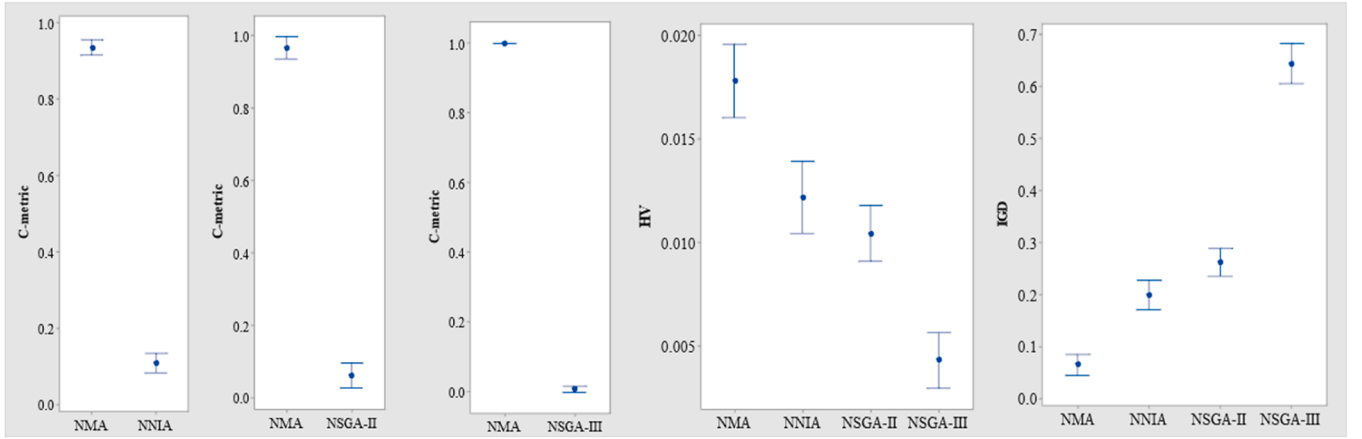


Fig. 19. Mean plots with 95 % CI in terms of C-metric, HV and IGD results of four algorithms.

time, total energy consumption, and the weighted average time of job priorities. The DFJSPJP can assist production managers in dealing with scheduling issues involving job priorities, which are widespread in the real world but have been completely neglected in theoretical research. We propose an effective NMA to solve the DFJSPJP. In the NMA, an CEM was constructed by sufficiently making use of the information and structure of the DFJSPJP, which can guarantee the NMA to obtain a high-quality initial population; and a local search operator (LSO) with two methods is designed to enhance the convergence speed of the NMA.

In the experimental section, we first construct 60 benchmark instances based on some classic FJSP benchmark instances to test the algorithm's performance. Fairly, the critical parameters of the NMA and the other three comparative algorithms (NNIA, NSGA-II, and NSGA-III) are calibrated using the Taguchi method. Additionally, we select 60 large, medium, and small-scale instances to validate the effectiveness of the proposed LSO. Furthermore, extensive comparisons between our proposed NMA and three well-known algorithms are conducted based on the 60 constructed benchmark instances. From these experimental results, it is evident that the NMA, evaluated based on C-metric, IGD, and HV evaluation metrics, outperforms the other three comparative algorithms significantly.

Our research provides significant social value in both theoretical and practical aspects. From a theoretical perspective, this study proposes a novel research direction for DFJSP by first incorporating job priority in distributed production environments. Furthermore, the proposed NMA algorithm and benchmark instances can serve as a reference for other researchers to design multi-objective optimization algorithms with local search operators and to test their algorithms using our benchmark instances. From the practical point of view, our proposed DFJSPJP model can effectively distribute the production of high-priority jobs. As a result, companies can improve productivity by ensuring that critical orders are delivered on time without compromising the overall schedule. In addition, the resulting scheduling scheme helps to address

the problem of workers working overtime to meet deadlines, thereby reducing worker overload and the safety accidents it may cause.

In the future work, some advanced algorithms can be introduced to enhance the performance of the algorithm, such as the latest meta-heuristic algorithms (e.g., flying fox optimizer) and reinforcement learning. Furthermore, the shop scheduling problem with job priority can be extended to dynamic scheduling scenarios (e.g., considering machine failures with planned outages).

#### CRediT authorship contribution statement

**Ao He:** Writing – original draft, Validation, Supervision, Investigation, Data curation, Conceptualization. **Xiahui Liu:** Writing – review & editing, Funding acquisition. **Guiliang Gong:** Writing – review & editing. **Zhipeng Yuan:** Writing – review & editing. **Hongbo Huang:** Writing – review & editing. **Yang Zhou:** Conceptualization. **Jie Li:** Data curation.

#### Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

#### Acknowledgement

This work was supported by the Hunan Provincial Department of Education Project (grant number: 23A0224); the Natural Science Foundation of Hunan Province of China (grant number: 2025JJ60371); the Changsha Soft Science Research Program (grant number: kh2302004); and the National Nature Science Foundation of China (grant number: 72001217).

#### Appendix

Fig. A1, Table A1, Table A2, Table A3, Table A4.

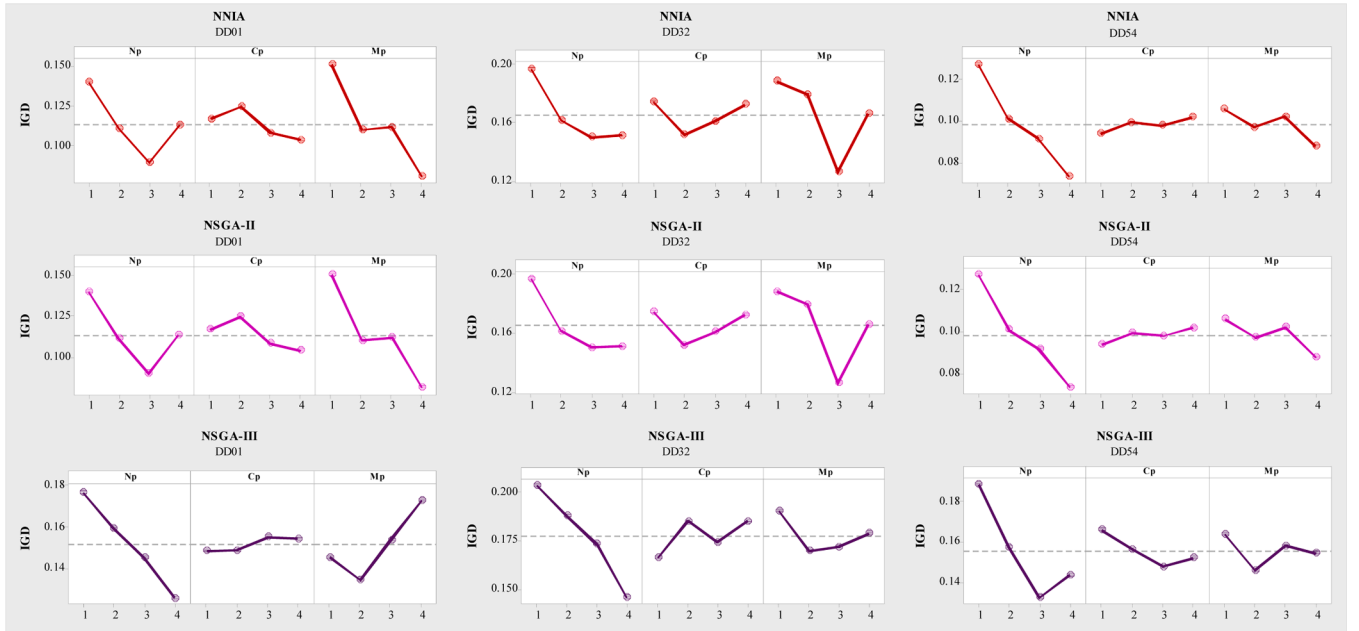


Fig. A1. Main effect of key parameter of comparison algorithms.

Table A1

C-metric, HV and IGD comparisons of NMA and NMA1.

| Benchmarks | Size<br>$n \times f \times w \times m$ | NMA(A) |        | NMA1(B) |        | HV     |        | IGD    |        |
|------------|----------------------------------------|--------|--------|---------|--------|--------|--------|--------|--------|
|            |                                        | C(A,B) | C(B,A) | C(A,B)  | C(B,A) | MA     | MA1    | MA     | MA1    |
| DD01       | 20×2×2×3                               | 1.0000 | 0.0000 | 1.0000  | 0.0000 | 0.0251 | 0.0093 | 0.0000 | 0.4367 |
| DD02       | 20×2×2×3                               | 1.0000 | 0.0000 | 1.0000  | 0.0000 | 0.0167 | 0.0051 | 0.0000 | 0.4481 |
| DD03       | 20×2×2×3                               | 1.0000 | 0.0000 | 1.0000  | 0.0000 | 0.0290 | 0.0112 | 0.0000 | 0.4906 |
| DD04       | 20×2×2×3                               | 1.0000 | 0.0000 | 1.0000  | 0.0000 | 0.0146 | 0.0050 | 0.0000 | 0.5083 |
| DD05       | 20×2×2×3                               | 1.0000 | 0.0000 | 1.0000  | 0.0000 | 0.0149 | 0.0066 | 0.0000 | 0.4028 |
| DD06       | 10×2×2×3                               | 1.0000 | 0.0000 | 1.0000  | 0.0000 | 0.0060 | 0.0024 | 0.0000 | 0.4929 |
| DD07       | 10×2×2×3                               | 1.0000 | 0.0000 | 1.0000  | 0.0000 | 0.0096 | 0.0042 | 0.0000 | 0.3820 |
| DD08       | 10×2×2×3                               | 1.0000 | 0.0000 | 1.0000  | 0.0000 | 0.0063 | 0.0020 | 0.0000 | 0.5265 |
| DD09       | 10×2×2×3                               | 1.0000 | 0.0000 | 1.0000  | 0.0000 | 0.0133 | 0.0041 | 0.0000 | 0.4931 |
| DD10       | 10×2×2×3                               | 1.0000 | 0.0000 | 1.0000  | 0.0000 | 0.0108 | 0.0041 | 0.0000 | 0.3957 |
| DD11       | 10×2×2×3                               | 1.0000 | 0.0000 | 1.0000  | 0.0000 | 0.0084 | 0.0021 | 0.0000 | 0.5268 |
| DD12       | 10×2×2×3                               | 1.0000 | 0.0000 | 1.0000  | 0.0000 | 0.0060 | 0.0016 | 0.0000 | 0.5037 |
| DD13       | 10×2×2×3                               | 0.8333 | 0.5000 | 1.0000  | 0.0000 | 0.0026 | 0.0011 | 0.0000 | 0.4968 |
| DD14       | 10×2×2×3                               | 1.0000 | 0.0000 | 1.0000  | 0.0000 | 0.0089 | 0.0027 | 0.0000 | 0.4207 |
| DD15       | 10×2×2×3                               | 1.0000 | 0.0000 | 1.0000  | 0.0000 | 0.0089 | 0.0042 | 0.0000 | 0.3286 |
| DD16       | 10×2×2×3                               | 1.0000 | 0.2083 | 1.0000  | 0.0000 | 0.0109 | 0.0066 | 0.0000 | 0.2741 |
| DD17       | 10×2×2×3                               | 1.0000 | 0.0000 | 1.0000  | 0.0000 | 0.0097 | 0.0010 | 0.0000 | 0.8770 |
| DD18       | 10×2×2×3                               | 1.0000 | 0.0000 | 1.0000  | 0.0000 | 0.0026 | 0.0003 | 0.0000 | 0.7952 |
| DD19       | 10×2×2×3                               | 1.0000 | 0.0000 | 1.0000  | 0.0000 | 0.0043 | 0.0014 | 0.0000 | 0.4478 |
| DD20       | 10×2×2×3                               | 0.8750 | 0.0357 | 1.0000  | 0.0000 | 0.0072 | 0.0033 | 0.0000 | 0.3404 |
| DD21       | 20×3×3×5                               | 1.0000 | 0.0000 | 1.0000  | 0.0000 | 0.0112 | 0.0058 | 0.0000 | 0.2599 |
| DD22       | 20×3×3×5                               | 1.0000 | 0.0435 | 1.0000  | 0.0000 | 0.0042 | 0.0026 | 0.0106 | 0.2824 |
| DD23       | 20×3×3×5                               | 1.0000 | 0.0000 | 1.0000  | 0.0000 | 0.0095 | 0.0027 | 0.0000 | 0.6565 |
| DD24       | 15×3×3×5                               | 1.0000 | 0.0000 | 1.0000  | 0.0000 | 0.0096 | 0.0035 | 0.0000 | 0.5381 |
| DD25       | 15×3×3×5                               | 1.0000 | 0.0000 | 1.0000  | 0.0000 | 0.0035 | 0.0009 | 0.0000 | 0.7161 |
| DD26       | 15×3×3×5                               | 1.0000 | 0.0952 | 1.0000  | 0.0000 | 0.0057 | 0.0022 | 0.0000 | 0.3786 |
| DD27       | 15×3×3×5                               | 1.0000 | 0.0000 | 1.0000  | 0.0000 | 0.0066 | 0.0034 | 0.0389 | 0.3369 |
| DD28       | 20×3×3×5                               | 1.0000 | 0.0000 | 1.0000  | 0.0000 | 0.0094 | 0.0040 | 0.0000 | 0.3877 |
| DD29       | 20×3×3×5                               | 1.0000 | 0.0000 | 1.0000  | 0.0000 | 0.0078 | 0.0015 | 0.0000 | 0.7366 |
| DD30       | 20×3×3×5                               | 1.0000 | 0.0000 | 1.0000  | 0.0000 | 0.0018 | 0.0000 | 0.0000 | 0.7220 |
| DD31       | 20×3×3×5                               | 1.0000 | 0.0000 | 1.0000  | 0.0000 | 0.0076 | 0.0030 | 0.0000 | 0.3172 |
| DD32       | 20×3×3×5                               | 1.0000 | 0.0000 | 1.0000  | 0.0000 | 0.0107 | 0.0037 | 0.0000 | 0.5313 |
| DD33       | 15×3×3×5                               | 1.0000 | 0.0000 | 1.0000  | 0.0000 | 0.0046 | 0.0007 | 0.0000 | 0.8361 |
| DD34       | 15×3×3×5                               | 1.0000 | 0.0000 | 1.0000  | 0.0000 | 0.0070 | 0.0017 | 0.0000 | 0.6413 |
| DD35       | 15×3×3×5                               | 1.0000 | 0.0000 | 1.0000  | 0.0000 | 0.0081 | 0.0019 | 0.0000 | 0.6567 |
| DD36       | 15×3×3×5                               | 1.0000 | 0.0000 | 1.0000  | 0.0000 | 0.0057 | 0.0013 | 0.0000 | 0.6522 |
| DD37       | 15×3×3×5                               | 0.6667 | 0.3846 | 1.0000  | 0.0000 | 0.0005 | 0.0003 | 0.0087 | 0.3837 |
| DD38       | 15×3×3×5                               | 1.0000 | 0.0000 | 1.0000  | 0.0000 | 0.0081 | 0.0017 | 0.0000 | 0.7840 |
| DD39       | 15×3×3×5                               | 1.0000 | 0.0000 | 1.0000  | 0.0000 | 0.0099 | 0.0021 | 0.0000 | 0.7660 |
| DD40       | 15×4×4×7                               | 1.0000 | 0.0000 | 1.0000  | 0.0000 | 0.0106 | 0.0040 | 0.0000 | 0.5341 |

(continued on next page)

Table A1 (continued)

| Benchmarks | Size<br>$n \times f \times w \times m$ | NMA(A) | NMA1(B) | HV     |        | IGD    |        |
|------------|----------------------------------------|--------|---------|--------|--------|--------|--------|
|            |                                        | C(A,B) | C(B,A)  | MA     | MA1    | MA     | MA1    |
| DD41       | 15×4 × 4 × 7                           | 1.0000 | 0.0000  | 0.0034 | 0.0009 | 0.0000 | 0.5362 |
| DD42       | 20×4 × 4 × 7                           | 1.0000 | 0.0000  | 0.0041 | 0.0010 | 0.0000 | 0.6027 |
| DD43       | 20×4 × 4 × 7                           | 1.0000 | 0.0000  | 0.0004 | 0.0001 | 0.0000 | 0.6595 |
| DD44       | 20×4 × 4 × 7                           | 1.0000 | 0.0000  | 0.0025 | 0.0000 | 0.0000 | 0.7222 |
| DD45       | 20×4 × 4 × 7                           | 1.0000 | 0.0370  | 0.0031 | 0.0015 | 0.0169 | 0.2503 |
| DD46       | 20×4 × 4 × 7                           | 1.0000 | 0.0000  | 0.0026 | 0.0005 | 0.0000 | 0.6167 |
| DD47       | 20×4 × 4 × 7                           | 1.0000 | 0.0000  | 0.0037 | 0.0009 | 0.0000 | 0.6326 |
| DD48       | 30×4 × 4 × 7                           | 1.0000 | 0.0000  | 0.0056 | 0.0008 | 0.0000 | 0.6263 |
| DD49       | 30×4 × 4 × 7                           | 1.0000 | 0.0000  | 0.0062 | 0.0016 | 0.0000 | 0.5899 |
| DD50       | 30×4 × 4 × 7                           | 1.0000 | 0.0000  | 0.0038 | 0.0015 | 0.0147 | 0.4127 |
| DD51       | 30×4 × 4 × 7                           | 1.0000 | 0.0333  | 0.0061 | 0.0019 | 0.0000 | 0.4719 |
| DD52       | 30×4 × 4 × 7                           | 1.0000 | 0.0000  | 0.0060 | 0.0014 | 0.0000 | 0.5713 |
| DD53       | 20×4 × 4 × 7                           | 1.0000 | 0.0000  | 0.0040 | 0.0009 | 0.0000 | 0.7256 |
| DD54       | 20×4 × 4 × 7                           | 1.0000 | 0.0000  | 0.0072 | 0.0036 | 0.0000 | 0.4601 |
| DD55       | 20×4 × 4 × 7                           | 1.0000 | 0.0000  | 0.0025 | 0.0004 | 0.0000 | 0.7719 |
| DD56       | 30×4 × 4 × 7                           | 1.0000 | 0.0000  | 0.0062 | 0.0012 | 0.0000 | 0.6148 |
| DD57       | 30×4 × 4 × 7                           | 1.0000 | 0.0000  | 0.0034 | 0.0005 | 0.0000 | 0.5768 |
| DD58       | 30×4 × 4 × 7                           | 1.0000 | 0.0000  | 0.0049 | 0.0009 | 0.0000 | 0.5281 |
| DD59       | 30×4 × 4 × 7                           | 1.0000 | 0.0000  | 0.0034 | 0.0011 | 0.0485 | 0.4135 |
| DD60       | 30×4 × 4 × 7                           | 1.0000 | 0.0000  | 0.0096 | 0.0021 | 0.0000 | 0.5897 |

Table A2

C-metric of the compared algorithms under the stopping standard of 100 iterations.

| Benchmarks | Size<br>$n \times f \times w \times m$ | D:NMA<br>C(D,D1) | D1:NNIA<br>C(D1,D) | D:NMA<br>C(D,D2) | D2:NSGA-II<br>C(D2,D) | D:NMA<br>C(D,D3) | D3:NSGA-III<br>C(D3,D) |
|------------|----------------------------------------|------------------|--------------------|------------------|-----------------------|------------------|------------------------|
|            |                                        |                  |                    |                  |                       |                  |                        |
| DD01       | 20×2 × 2 × 3                           | 0.8824           | 0.0000             | 0.6053           | 0.2143                | 1.0000           | 0.0000                 |
| DD02       | 20×2 × 2 × 3                           | 1.0000           | 0.0000             | 1.0000           | 0.0667                | 1.0000           | 0.0000                 |
| DD03       | 20×2 × 2 × 3                           | 1.0000           | 0.0000             | 1.0000           | 0.0000                | 1.0000           | 0.0000                 |
| DD04       | 20×2 × 2 × 3                           | 0.0333           | 0.7692             | 0.5556           | 0.5385                | 1.0000           | 0.0000                 |
| DD05       | 20×2 × 2 × 3                           | 0.7570           | 0.0000             | 0.7778           | 0.4375                | 1.0000           | 0.0000                 |
| DD06       | 10×2 × 2 × 3                           | 0.4615           | 0.3077             | 1.0000           | 0.1538                | 1.0000           | 0.0000                 |
| DD07       | 10×2 × 2 × 3                           | 1.0000           | 0.0000             | 1.0000           | 0.0000                | 1.0000           | 0.0000                 |
| DD08       | 10×2 × 2 × 3                           | 1.0000           | 0.0000             | 1.0000           | 0.0000                | 1.0000           | 0.0000                 |
| DD09       | 10×2 × 2 × 3                           | 1.0000           | 0.0000             | 1.0000           | 0.0000                | 1.0000           | 0.0000                 |
| DD10       | 10×2 × 2 × 3                           | 0.8200           | 0.2000             | 0.9655           | 0.1000                | 1.0000           | 0.0000                 |
| DD11       | 10×2 × 2 × 3                           | 1.0000           | 0.0000             | 1.0000           | 0.0000                | 1.0000           | 0.0000                 |
| DD12       | 10×2 × 2 × 3                           | 0.7027           | 0.3636             | 0.8800           | 0.0000                | 1.0000           | 0.0000                 |
| DD13       | 10×2 × 2 × 3                           | 0.9259           | 0.1000             | 1.0000           | 0.0000                | 1.0000           | 0.0000                 |
| DD14       | 10×2 × 2 × 3                           | 1.0000           | 0.0000             | 1.0000           | 0.0000                | 1.0000           | 0.0000                 |
| DD15       | 10×2 × 2 × 3                           | 0.9792           | 0.0000             | 1.0000           | 0.0000                | 1.0000           | 0.0000                 |
| DD16       | 10×2 × 2 × 3                           | 0.3824           | 0.5000             | 0.8182           | 0.5000                | 1.0000           | 0.0000                 |
| DD17       | 10×2 × 2 × 3                           | 1.0000           | 0.0000             | 1.0000           | 0.0000                | 1.0000           | 0.0000                 |
| DD18       | 10×2 × 2 × 3                           | 0.9048           | 0.0000             | 1.0000           | 0.0000                | 1.0000           | 0.0000                 |
| DD19       | 10×2 × 2 × 3                           | 0.0000           | 0.8333             | 0.8776           | 0.0000                | 1.0000           | 0.0000                 |
| DD20       | 10×2 × 2 × 3                           | 0.9500           | 0.0000             | 0.9688           | 0.1111                | 1.0000           | 0.0000                 |
| DD21       | 20×3 × 3 × 5                           | 1.0000           | 0.0000             | 1.0000           | 0.0000                | 1.0000           | 0.0000                 |
| DD22       | 20×3 × 3 × 5                           | 0.9167           | 0.0556             | 0.9020           | 0.3333                | 1.0000           | 0.0000                 |
| DD23       | 20×3 × 3 × 5                           | 1.0000           | 0.0000             | 1.0000           | 0.1538                | 1.0000           | 0.0000                 |
| DD24       | 15×3 × 3 × 5                           | 1.0000           | 0.0000             | 1.0000           | 0.0000                | 1.0000           | 0.0000                 |
| DD25       | 15×3 × 3 × 5                           | 0.9636           | 0.1429             | 0.8814           | 0.0000                | 1.0000           | 0.0000                 |
| DD26       | 15×3 × 3 × 5                           | 1.0000           | 0.0000             | 1.0000           | 0.0357                | 1.0000           | 0.0000                 |
| DD27       | 15×3 × 3 × 5                           | 0.8222           | 0.0000             | 0.8413           | 0.1765                | 1.0000           | 0.0000                 |
| DD28       | 20×3 × 3 × 5                           | 1.0000           | 0.0000             | 1.0000           | 0.0000                | 1.0000           | 0.0000                 |
| DD29       | 20×3 × 3 × 5                           | 1.0000           | 0.0000             | 1.0000           | 0.0000                | 1.0000           | 0.0000                 |
| DD30       | 20×3 × 3 × 5                           | 1.0000           | 0.0000             | 1.0000           | 0.0000                | 1.0000           | 0.0000                 |
| DD31       | 20×3 × 3 × 5                           | 1.0000           | 0.0000             | 0.9464           | 0.0667                | 1.0000           | 0.0000                 |
| DD32       | 20×3 × 3 × 5                           | 1.0000           | 0.0000             | 1.0000           | 0.0000                | 1.0000           | 0.0000                 |
| DD33       | 15×3 × 3 × 5                           | 0.9167           | 0.0000             | 1.0000           | 0.0000                | 1.0000           | 0.0000                 |
| DD34       | 15×3 × 3 × 5                           | 1.0000           | 0.0000             | 1.0000           | 0.0000                | 1.0000           | 0.0000                 |
| DD35       | 15×3 × 3 × 5                           | 1.0000           | 0.2353             | 1.0000           | 0.0000                | 1.0000           | 0.0000                 |
| DD36       | 15×3 × 3 × 5                           | 1.0000           | 0.0000             | 1.0000           | 0.0000                | 1.0000           | 0.0000                 |
| DD37       | 15×3 × 3 × 5                           | 0.8824           | 0.3158             | 0.8431           | 0.3158                | 1.0000           | 0.0000                 |
| DD38       | 15×3 × 3 × 5                           | 1.0000           | 0.0000             | 1.0000           | 0.0000                | 1.0000           | 0.0000                 |
| DD39       | 15×3 × 3 × 5                           | 1.0000           | 0.0000             | 1.0000           | 0.0000                | 1.0000           | 0.0000                 |
| DD40       | 15×4 × 4 × 7                           | 0.9048           | 0.4286             | 1.0000           | 0.1786                | 1.0000           | 0.0000                 |
| DD41       | 15×4 × 4 × 7                           | 1.0000           | 0.0000             | 1.0000           | 0.0000                | 1.0000           | 0.0000                 |
| DD42       | 20×4 × 4 × 7                           | 1.0000           | 0.0000             | 1.0000           | 0.0000                | 1.0000           | 0.0000                 |
| DD43       | 20×4 × 4 × 7                           | 1.0000           | 0.0556             | 1.0000           | 0.0000                | 1.0000           | 0.0000                 |
| DD44       | 20×4 × 4 × 7                           | 1.0000           | 0.0000             | 1.0000           | 0.0000                | 1.0000           | 0.0000                 |
| DD45       | 20×4 × 4 × 7                           | 0.9057           | 0.0000             | 0.8571           | 0.0000                | 1.0000           | 0.0000                 |

(continued on next page)

Table A2 (continued)

| Benchmarks         | Size<br>$n \times f \times w \times m$ | D:NMA<br>C(D,D1) | D1:NNIA<br>C(D1,D) | D:NMA<br>C(D,D2) | D2:NSGA-II<br>C(D2,D) | D:NMA<br>C(D,D3) | D3:NSGA-III<br>C(D3,D) |
|--------------------|----------------------------------------|------------------|--------------------|------------------|-----------------------|------------------|------------------------|
| DD46               | 20×4 × 4 × 7                           | <b>1.0000</b>    | 0.0000             | <b>1.0000</b>    | 0.0000                | <b>1.0000</b>    | 0.0000                 |
| DD47               | 20×4 × 4 × 7                           | <b>1.0000</b>    | 0.0000             | <b>1.0000</b>    | 0.0000                | <b>1.0000</b>    | 0.0000                 |
| DD48               | 30×4 × 4 × 7                           | <b>1.0000</b>    | 0.0000             | <b>1.0000</b>    | 0.0870                | <b>1.0000</b>    | 0.0000                 |
| DD49               | 30×4 × 4 × 7                           | <b>1.0000</b>    | 0.0000             | <b>1.0000</b>    | 0.0000                | <b>1.0000</b>    | 0.0000                 |
| DD50               | 30×4 × 4 × 7                           | <b>1.0000</b>    | 0.0000             | <b>1.0000</b>    | 0.0000                | <b>1.0000</b>    | 0.0000                 |
| DD51               | 30×4 × 4 × 7                           | <b>1.0000</b>    | 0.0000             | <b>1.0000</b>    | 0.0000                | <b>1.0000</b>    | 0.0000                 |
| DD52               | 30×4 × 4 × 7                           | <b>1.0000</b>    | 0.0526             | <b>1.0000</b>    | 0.0000                | <b>1.0000</b>    | 0.0000                 |
| DD53               | 20×4 × 4 × 7                           | <b>1.0000</b>    | 0.0000             | <b>1.0000</b>    | 0.0000                | <b>1.0000</b>    | 0.0000                 |
| DD54               | 20×4 × 4 × 7                           | <b>0.8095</b>    | 0.2143             | <b>0.8305</b>    | 0.0000                | <b>1.0000</b>    | 0.0000                 |
| DD55               | 20×4 × 4 × 7                           | <b>1.0000</b>    | 0.0000             | <b>1.0000</b>    | 0.0400                | <b>1.0000</b>    | 0.0000                 |
| DD56               | 30×4 × 4 × 7                           | <b>1.0000</b>    | 0.0000             | <b>1.0000</b>    | 0.0000                | <b>1.0000</b>    | 0.0000                 |
| DD57               | 30×4 × 4 × 7                           | <b>1.0000</b>    | 0.1000             | <b>1.0000</b>    | 0.1500                | <b>1.0000</b>    | 0.0000                 |
| DD58               | 30×4 × 4 × 7                           | <b>1.0000</b>    | 0.0000             | <b>1.0000</b>    | 0.0000                | <b>1.0000</b>    | 0.0000                 |
| DD59               | 30×4 × 4 × 7                           | <b>1.0000</b>    | 0.3333             | <b>1.0000</b>    | 0.0000                | <b>1.0000</b>    | 0.0000                 |
| DD60               | 30×4 × 4 × 7                           | <b>1.0000</b>    | 0.0000             | <b>1.0000</b>    | 0.0000                | <b>1.0000</b>    | 0.0000                 |
| Win numbers (Rate) |                                        | 57(95 %)         | 3(5 %)             | 60(100 %)        | 0(0 %)                | 60(100 %)        | 0(0 %)                 |

Table A3

HV and IGD value of the compared algorithms under the stopping standard of 100 iterations.

| Benchmarks | HV            |               |         |          | IGD           |               |               |          |
|------------|---------------|---------------|---------|----------|---------------|---------------|---------------|----------|
|            | MA            | NNIA          | NSGA-II | NSGA-III | NMA           | NNIA          | NSGA-II       | NSGA-III |
| DD01       | <b>0.0145</b> | 0.0117        | 0.0093  | 0.0010   | <b>0.0867</b> | 0.1017        | 0.1032        | 0.6947   |
| DD02       | <b>0.0264</b> | 0.0203        | 0.0176  | 0.0029   | <b>0.1017</b> | 0.2114        | 0.2402        | 0.7412   |
| DD03       | <b>0.0453</b> | 0.0309        | 0.0274  | 0.0069   | 0.3563        | <b>0.1773</b> | 0.1908        | 0.5177   |
| DD04       | <b>0.0212</b> | 0.0184        | 0.0173  | 0.0044   | 0.1465        | <b>0.0488</b> | 0.0716        | 0.5134   |
| DD05       | <b>0.0220</b> | 0.0164        | 0.0142  | 0.0030   | <b>0.1146</b> | 0.1285        | 0.2455        | 0.6324   |
| DD06       | <b>0.0181</b> | 0.0160        | 0.0128  | 0.0018   | <b>0.0552</b> | 0.1228        | 0.2216        | 0.8548   |
| DD07       | <b>0.0237</b> | 0.0198        | 0.0150  | 0.0022   | 0.4399        | <b>0.1336</b> | 0.1392        | 0.7193   |
| DD08       | <b>0.0472</b> | 0.0392        | 0.0308  | 0.0030   | <b>0.0000</b> | 0.1191        | 0.3475        | 0.9769   |
| DD09       | <b>0.0276</b> | 0.0201        | 0.0142  | 0.0022   | <b>0.0000</b> | 0.2892        | 0.4699        | 0.9768   |
| DD10       | 0.0255        | <b>0.0277</b> | 0.0221  | 0.0036   | 0.1940        | <b>0.0048</b> | 0.1657        | 0.6829   |
| DD11       | <b>0.0338</b> | 0.0219        | 0.0191  | 0.0019   | <b>0.0000</b> | 0.2469        | 0.3358        | 1.0455   |
| DD12       | <b>0.0297</b> | 0.0270        | 0.0186  | 0.0021   | 0.1451        | <b>0.0399</b> | 0.1553        | 0.7624   |
| DD13       | <b>0.0223</b> | 0.0201        | 0.0161  | 0.0018   | <b>0.0333</b> | 0.0387        | 0.1661        | 0.9550   |
| DD14       | <b>0.0247</b> | 0.0211        | 0.0174  | 0.0049   | <b>0.0291</b> | 0.0966        | 0.2448        | 0.7284   |
| DD15       | <b>0.0309</b> | 0.0281        | 0.0208  | 0.0044   | <b>0.0172</b> | 0.1616        | 0.2634        | 0.7988   |
| DD16       | <b>0.0196</b> | 0.0136        | 0.0114  | 0.0020   | 0.3079        | <b>0.0761</b> | 0.1271        | 0.5862   |
| DD17       | <b>0.0268</b> | 0.0178        | 0.0134  | 0.0017   | <b>0.0000</b> | 0.2311        | 0.4051        | 1.0022   |
| DD18       | <b>0.0400</b> | 0.0297        | 0.0237  | 0.0036   | 0.1697        | <b>0.1172</b> | 0.2360        | 0.9029   |
| DD19       | <b>0.0211</b> | 0.0181        | 0.0102  | 0.0011   | <b>0.0970</b> | 0.1608        | 0.3938        | 1.0072   |
| DD20       | <b>0.0363</b> | 0.0305        | 0.0292  | 0.0044   | 0.4069        | 0.0710        | <b>0.0594</b> | 0.5803   |
| DD21       | <b>0.0153</b> | 0.0111        | 0.0112  | 0.0039   | <b>0.0000</b> | 0.2588        | 0.1824        | 0.6301   |
| DD22       | <b>0.0121</b> | 0.0103        | 0.0117  | 0.0050   | 0.1155        | 0.1304        | <b>0.0517</b> | 0.3109   |
| DD23       | <b>0.0166</b> | 0.0107        | 0.0116  | 0.0021   | <b>0.0000</b> | 0.3105        | 0.2552        | 0.8890   |
| DD24       | <b>0.0209</b> | 0.0122        | 0.0107  | 0.0038   | <b>0.0000</b> | 0.3325        | 0.4429        | 0.7634   |
| DD25       | <b>0.0173</b> | 0.0134        | 0.0096  | 0.0036   | 0.3151        | <b>0.1432</b> | 0.1381        | 0.4241   |
| DD26       | <b>0.0187</b> | 0.0133        | 0.0106  | 0.0021   | <b>0.0000</b> | 0.2636        | 0.3229        | 0.8065   |
| DD27       | <b>0.0109</b> | 0.0092        | 0.0081  | 0.0022   | 0.1114        | <b>0.0907</b> | 0.1219        | 0.5673   |
| DD28       | <b>0.0179</b> | 0.0110        | 0.0102  | 0.0032   | <b>0.0000</b> | 0.4122        | 0.4150        | 0.8369   |
| DD29       | <b>0.0270</b> | 0.0149        | 0.0144  | 0.0048   | <b>0.0000</b> | 0.5253        | 0.4600        | 0.7979   |
| DD30       | <b>0.0212</b> | 0.0155        | 0.0138  | 0.0059   | 0.3280        | <b>0.1349</b> | 0.1919        | 0.4768   |
| DD31       | <b>0.0125</b> | 0.0093        | 0.0083  | 0.0028   | <b>0.1023</b> | 0.1899        | 0.2065        | 0.5972   |
| DD32       | <b>0.0134</b> | 0.0079        | 0.0080  | 0.0013   | <b>0.0853</b> | 0.2051        | 0.1567        | 0.7656   |
| DD33       | <b>0.0193</b> | 0.0131        | 0.0087  | 0.0023   | <b>0.0595</b> | 0.2715        | 0.4704        | 0.8883   |
| DD34       | <b>0.0190</b> | 0.0130        | 0.0099  | 0.0038   | <b>0.0000</b> | 0.2324        | 0.3225        | 0.6934   |
| DD35       | <b>0.0205</b> | 0.0139        | 0.0097  | 0.0024   | <b>0.0000</b> | 0.2313        | 0.4144        | 0.8412   |
| DD36       | <b>0.0219</b> | 0.0151        | 0.0143  | 0.0059   | <b>0.1055</b> | 0.2398        | 0.2407        | 0.6558   |
| DD37       | <b>0.0094</b> | 0.0078        | 0.0067  | 0.0018   | 0.1105        | <b>0.0595</b> | 0.1189        | 0.5176   |
| DD38       | <b>0.0140</b> | 0.0074        | 0.0078  | 0.0019   | <b>0.0000</b> | 0.4882        | 0.4140        | 0.7776   |
| DD39       | <b>0.0169</b> | 0.0099        | 0.0085  | 0.0016   | <b>0.0000</b> | 0.4019        | 0.3839        | 0.8855   |
| DD40       | <b>0.0201</b> | 0.0141        | 0.0112  | 0.0024   | <b>0.0555</b> | 0.1315        | 0.2179        | 0.7552   |
| DD41       | <b>0.0121</b> | 0.0072        | 0.0059  | 0.0023   | <b>0.0000</b> | 0.3818        | 0.4654        | 0.7451   |
| DD42       | <b>0.0101</b> | 0.0059        | 0.0065  | 0.0025   | <b>0.0000</b> | 0.2996        | 0.2231        | 0.5952   |
| DD43       | <b>0.0099</b> | 0.0075        | 0.0068  | 0.0031   | <b>0.0101</b> | 0.1490        | 0.2818        | 0.6101   |
| DD44       | <b>0.0092</b> | 0.0063        | 0.0064  | 0.0030   | <b>0.0946</b> | 0.1601        | 0.1768        | 0.4379   |
| DD45       | <b>0.0123</b> | 0.0111        | 0.0102  | 0.0058   | <b>0.0644</b> | 0.0747        | 0.0980        | 0.3717   |
| DD46       | <b>0.0065</b> | 0.0040        | 0.0033  | 0.0011   | <b>0.0000</b> | 0.2704        | 0.3625        | 0.6714   |
| DD47       | <b>0.0154</b> | 0.0079        | 0.0083  | 0.0024   | <b>0.0000</b> | 0.3166        | 0.3051        | 0.7411   |
| DD48       | <b>0.0162</b> | 0.0071        | 0.0078  | 0.0023   | <b>0.0000</b> | 0.4128        | 0.3650        | 0.7203   |
| DD49       | <b>0.0161</b> | 0.0084        | 0.0089  | 0.0032   | <b>0.0000</b> | 0.3138        | 0.3559        | 0.7371   |

(continued on next page)

Table A3 (continued)

| Benchmarks         | HV            |        |         |          | IGD           |               |         |          |
|--------------------|---------------|--------|---------|----------|---------------|---------------|---------|----------|
|                    | MA            | NNIA   | NSGA-II | NSGA-III | NMA           | NNIA          | NSGA-II | NSGA-III |
| DD50               | <b>0.0151</b> | 0.0091 | 0.0094  | 0.0036   | <b>0.0000</b> | 0.3214        | 0.2839  | 0.6156   |
| DD51               | <b>0.0126</b> | 0.0068 | 0.0079  | 0.0025   | <b>0.0000</b> | 0.3907        | 0.3536  | 0.7180   |
| DD52               | <b>0.0157</b> | 0.0085 | 0.0081  | 0.0028   | <b>0.0000</b> | 0.2801        | 0.3448  | 0.6325   |
| DD53               | <b>0.0148</b> | 0.0072 | 0.0079  | 0.0033   | <b>0.0000</b> | 0.4273        | 0.3920  | 0.6874   |
| DD54               | <b>0.0114</b> | 0.0106 | 0.0095  | 0.0039   | 0.0549        | <b>0.0478</b> | 0.1204  | 0.5123   |
| DD55               | <b>0.0101</b> | 0.0051 | 0.0060  | 0.0022   | <b>0.0000</b> | 0.4079        | 0.3103  | 0.6692   |
| DD56               | <b>0.0136</b> | 0.0058 | 0.0045  | 0.0011   | <b>0.0000</b> | 0.3958        | 0.5649  | 0.8676   |
| DD57               | <b>0.0090</b> | 0.0052 | 0.0052  | 0.0015   | <b>0.0000</b> | 0.2629        | 0.2603  | 0.6023   |
| DD58               | <b>0.0157</b> | 0.0104 | 0.0086  | 0.0041   | <b>0.0000</b> | 0.2059        | 0.2936  | 0.5516   |
| DD59               | <b>0.0104</b> | 0.0066 | 0.0064  | 0.0024   | <b>0.0000</b> | 0.3044        | 0.3997  | 0.6837   |
| DD60               | <b>0.0210</b> | 0.0105 | 0.0109  | 0.0045   | <b>0.0000</b> | 0.4181        | 0.3970  | 0.6591   |
| Win numbers (Rate) | 59(98 %)      | 1(2 %) | 0(0 %)  | 0(0 %)   | 46(76 %)      | 12(20 %)      | 2(4 %)  | 0(0 %)   |

Table A4

The runtime of the compared algorithms under the stopping standard of 100 iterations.

| Benchmarks | Runtime |                |         |          | Benchmarks | Runtime |                |         |                |
|------------|---------|----------------|---------|----------|------------|---------|----------------|---------|----------------|
|            | NMA     | NNIA           | NSGA-II | NSGA-III |            | NMA     | NNIA           | NSGA-II | NSGA-III       |
| DD01       | 1007.47 | <b>500.93</b>  | 983.65  | 838.31   | DD31       | 2123.52 | 1505.48        | 1723.56 | <b>1436.04</b> |
| DD02       | 991.50  | <b>497.47</b>  | 936.78  | 786.17   | DD32       | 2138.35 | <b>1310.42</b> | 1714.40 | 1432.94        |
| DD03       | 976.59  | <b>710.50</b>  | 931.04  | 788.50   | DD33       | 2419.23 | <b>1464.35</b> | 1990.74 | 1630.51        |
| DD04       | 990.18  | <b>704.77</b>  | 927.50  | 785.94   | DD34       | 2684.52 | <b>1498.84</b> | 1969.17 | 1630.39        |
| DD05       | 1011.93 | <b>702.99</b>  | 935.28  | 789.97   | DD35       | 2540.42 | <b>1453.15</b> | 1979.46 | 1638.42        |
| DD06       | 1026.70 | <b>691.06</b>  | 938.62  | 786.84   | DD36       | 2600.23 | <b>1487.10</b> | 1986.39 | 1655.44        |
| DD07       | 1049.29 | <b>497.23</b>  | 943.97  | 785.29   | DD37       | 2587.71 | <b>1488.90</b> | 1966.79 | 1643.20        |
| DD08       | 1134.24 | <b>694.74</b>  | 943.61  | 788.45   | DD38       | 2709.99 | <b>1485.28</b> | 1968.24 | 1653.21        |
| DD09       | 1152.72 | <b>500.83</b>  | 948.13  | 790.82   | DD39       | 2619.46 | <b>1481.69</b> | 1953.37 | 1646.65        |
| DD10       | 1108.41 | <b>698.68</b>  | 935.69  | 785.27   | DD40       | 2529.19 | <b>1488.33</b> | 1968.00 | 1640.02        |
| DD11       | 1043.77 | <b>548.16</b>  | 936.57  | 788.22   | DD41       | 4798.79 | <b>2346.00</b> | 3138.82 | 2594.06        |
| DD12       | 1060.61 | <b>702.36</b>  | 937.09  | 786.09   | DD42       | 4978.01 | <b>2381.21</b> | 3137.72 | 2623.01        |
| DD13       | 1081.60 | <b>702.74</b>  | 946.14  | 785.70   | DD43       | 5076.34 | <b>2342.56</b> | 3108.64 | 2580.53        |
| DD14       | 998.49  | <b>496.59</b>  | 955.55  | 782.27   | DD44       | 5085.35 | <b>2354.66</b> | 3121.83 | 2590.41        |
| DD15       | 1146.69 | <b>701.27</b>  | 935.99  | 791.39   | DD45       | 3576.15 | <b>1680.12</b> | 2268.37 | 1855.80        |
| DD16       | 1004.18 | <b>696.79</b>  | 937.34  | 779.46   | DD46       | 3337.79 | <b>1680.13</b> | 2285.59 | 1857.84        |
| DD17       | 1062.65 | <b>694.97</b>  | 938.86  | 790.07   | DD47       | 3157.39 | <b>1698.54</b> | 2259.59 | 1854.82        |
| DD18       | 1026.52 | <b>692.77</b>  | 947.63  | 785.75   | DD48       | 2706.46 | <b>1701.41</b> | 2219.16 | 1840.97        |
| DD19       | 992.35  | <b>688.37</b>  | 934.93  | 785.83   | DD49       | 2867.41 | <b>1691.38</b> | 2111.55 | 1842.20        |
| DD20       | 1011.42 | <b>699.86</b>  | 937.68  | 783.97   | DD50       | 2805.66 | <b>1682.63</b> | 2035.06 | 1844.74        |
| DD21       | 2871.42 | <b>1559.68</b> | 2084.92 | 1729.70  | DD51       | 2458.28 | <b>1254.42</b> | 2048.40 | 1850.81        |
| DD22       | 3015.85 | <b>1557.02</b> | 2088.94 | 1735.04  | DD52       | 2758.27 | <b>1672.42</b> | 2038.31 | 1837.95        |
| DD23       | 1569.40 | <b>909.01</b>  | 1292.19 | 1012.96  | DD53       | 2801.74 | <b>1239.51</b> | 1981.58 | 1846.56        |
| DD24       | 1664.89 | <b>890.85</b>  | 1420.51 | 1013.63  | DD54       | 3245.62 | <b>1663.13</b> | 1965.34 | 1848.57        |
| DD25       | 1459.53 | <b>907.49</b>  | 1525.96 | 1020.99  | DD55       | 3173.03 | <b>1671.04</b> | 1980.14 | 1852.75        |
| DD26       | 1544.89 | <b>907.47</b>  | 1272.34 | 1018.72  | DD56       | 2704.92 | <b>1663.72</b> | 1942.43 | 1761.08        |
| DD27       | 1488.58 | <b>906.80</b>  | 1262.40 | 1058.93  | DD57       | 2717.11 | <b>1662.81</b> | 1919.66 | 1709.81        |
| DD28       | 2047.34 | <b>1274.47</b> | 1735.74 | 1575.47  | DD58       | 2676.79 | <b>1670.19</b> | 1910.73 | 1698.46        |
| DD29       | 2106.56 | <b>1307.38</b> | 1729.93 | 1638.34  | DD59       | 2669.70 | <b>1614.24</b> | 1927.05 | 1715.41        |
| DD30       | 2155.17 | <b>1413.35</b> | 1733.54 | 1441.08  | DD60       | 2782.70 | <b>1657.03</b> | 1907.72 | 1710.04        |

Data availability

Data will be made available on request.

References

[1] Y. Zhang, H. Zhu, D. Tang, T. Zhou, Y. Gui, Dynamic job shop scheduling based on deep reinforcement learning for multi-agent manufacturing systems, Robot. Comput. Integr. Manuf. 78 (2022) 102412, <https://doi.org/10.1016/j.rcim.2022.102412>.

[2] J.-P. Huang, L. Gao, X.-Y. Li, An end-to-end deep reinforcement learning method based on graph neural network for distributed job-shop scheduling problem, Expert. Syst. Appl. 238 (2024) 121756, <https://doi.org/10.1016/j.eswa.2023.121756>.

[3] J. Xie, X. Li, L. Gao, L. Gui, A hybrid genetic tabu search algorithm for distributed flexible job shop scheduling problems, J. Manuf. Syst. 71 (2023) 82–94, <https://doi.org/10.1016/j.jmsy.2023.09.002>.

[4] Z. Zhang, Y. Fu, K. Gao, Q. Pan, M. Huang, A learning-driven multi-objective cooperative artificial bee colony algorithm for distributed flexible job shop scheduling problems with preventive maintenance and transportation operations, Comput. Ind. Eng. 196 (2024) 110484, <https://doi.org/10.1016/j.cie.2024.110484>.

[5] Y. Yuan, H. Xu, B. Wang, An improved NSGA-III procedure for evolutionary many-objective optimization, in: Proceedings of the 2014 Annual Conference on Genetic and Evolutionary Computation, ACM, 2014, pp. 661–668, <https://doi.org/10.1145/2576768>.

[6] G. Wang, X. Li, L. Gao, P. Li, An effective multi-objective whale swarm algorithm for energy-efficient scheduling of distributed welding flow shop, Ann. Oper. Res. (2022) 1–33, <https://doi.org/10.1007/s10479-021-03952-1>.

[7] F. Yu, C. Lu, J. Zhou, L. Yin, K. Wang, A knowledge-guided bi-population evolutionary algorithm for energy-efficient scheduling of distributed flexible job shop problem, Eng. Appl. Artif. Intell. 128 (2024) 107458, <https://doi.org/10.1016/j.engappai.2023.107458>.

[8] Z.-Q. Zhang, F.-C. Wu, B. Qian, R. Hu, L. Wang, H.-P. Jin, A Q-learning-based hyper-heuristic evolutionary algorithm for the distributed flexible job-shop scheduling problem with crane transportation, Expert. Syst. Appl. 234 (2023) 121050, <https://doi.org/10.1016/j.eswa.2023.121050>.

[9] S. Cao, R. Li, W. Gong, C. Lu, Inverse model and adaptive neighborhood search based cooperative optimizer for energy-efficient distributed flexible job shop

- scheduling, *Swarm. Evol. Comput.* 83 (2023) 101419, <https://doi.org/10.1016/j.swevo.2023.101419>.
- [10] Q. Luo, Q. Deng, G. Gong, L. Zhang, W. Han, K. Li, An efficient memetic algorithm for distributed flexible job shop scheduling problem with transfers, *Expert. Syst. Appl.* 160 (2020) 113721, <https://doi.org/10.1016/j.eswa.2020.113721>.
- [11] C. Luo, W. Gong, F. Ming, C. Lu, A Q-learning memetic algorithm for energy-efficient heterogeneous distributed assembly permutation flowshop scheduling considering priorities, *Swarm. Evol. Comput.* 85 (2024) 101497, <https://doi.org/10.1016/j.swevo.2024.101497>.
- [12] M. Liu, J. Lv, S. Du, Y. Deng, X. Shen, Y. Zhou, Multi-resource constrained flexible job shop scheduling problem with fixture-pallet combinatorial optimisation, *Comput. Ind. Eng.* 188 (2024) 109903, <https://doi.org/10.1016/j.cie.2024.109903>.
- [13] C. Destouet, H. Tlahig, B. Bettayeb, B. Mazari, Flexible job shop scheduling problem under Industry 5.0: a survey on human reintegration, environmental consideration and resilience improvement, *J. Manuf. Syst.* 67 (2023) 155–173, <https://doi.org/10.1016/j.jmsy.2023.01.004>.
- [14] Y. Xu, D. Wang, M. Zhang, M. Yang, C. Liang, Quantum particle swarm optimization with chaotic encoding schemes for flexible job-shop scheduling problem, *Swarm. Evol. Comput.* 93 (2025) 101836, <https://doi.org/10.1016/j.swevo.2024.101836>.
- [15] Y. Sang, J. Tan, Intelligent factory many-objective distributed flexible job shop collaborative scheduling method, *Comput. Ind. Eng.* 164 (2022) 107884, <https://doi.org/10.1016/j.cie.2021.107884>.
- [16] J. Tang, G. Gong, N. Peng, K. Zhu, D. Huang, Q. Luo, An effective memetic algorithm for distributed flexible job shop scheduling problem considering integrated sequencing flexibility, *Expert. Syst. Appl.* 242 (2024) 122734, <https://doi.org/10.1016/j.eswa.2023.122734>.
- [17] Z. Zhang, Y. Fu, K. Gao, H. Zhang, L. Wang, A cooperative evolutionary algorithm with simulated annealing for integrated scheduling of distributed flexible job shops and distribution, *Swarm. Evol. Comput.* 85 (2024) 101467, <https://doi.org/10.1016/j.swevo.2023.101467>.
- [18] C. Wang, M. Wei, Q. Liu, X. Zhang, X. Li, An improved adaptive hybrid algorithm for solving distributed flexible job shop scheduling problem, *Swarm. Evol. Comput.* 94 (2025) 101873, <https://doi.org/10.1016/j.swevo.2025.101873>.
- [19] L. De Giovanni, F. Pezzella, An improved genetic algorithm for the distributed and flexible job-shop scheduling problem, *Eur. J. Oper. Res.* 200 (2) (2010) 395–408, <https://doi.org/10.1016/j.ejor.2009.01.008>.
- [20] Y. Du, J.-Q. Li, C. Luo, L.-L. Meng, A hybrid estimation of distribution algorithm for distributed flexible job shop scheduling with crane transportations, *Swarm. Evol. Comput.* 62 (2021) 100861, <https://doi.org/10.1016/j.swevo.2021.100861>.
- [21] J.-Q. Li, P.-Y. Duan, J.-D. Cao, X.-P. Lin, Y.-Y. Han, A hybrid pareto-based Tabu search for the distributed flexible job shop scheduling problem with E/T criteria, *IEEE Access.* 6 (1) (2018) 58883–58897, <https://doi.org/10.1109/ACCESS.2018.2873401>.
- [22] B. Marzouki, O.B. Driss, K. Ghédira, Decentralized Tabu searches in multi agent system for distributed and flexible job shop scheduling problem, in: 2017 IEEE/ACS 14th International Conference on Computer Systems and Applications (AICCSA), IEEE, 2017, pp. 1019–1026, <https://doi.org/10.1109/AICCSA.2017.133>.
- [23] B. Marzouki, O.B. Driss, K. Ghédira, Solving distributed and flexible job shop scheduling problem using a chemical reaction optimization metaheuristic, *Procedia Comput. Sci.* 126 (1) (2018) 1424–1433, <https://doi.org/10.1016/j.procs.2018.08.114>.
- [24] X. Wu, X. Liu, An improved differential evolution algorithm for solving a distributed flexible job shop scheduling problem, in: 2018 IEEE 14th International Conference on Automation Science and Engineering (CASE), IEEE, 2018, pp. 968–973, <https://doi.org/10.1109/COASE.2018.8560396>.
- [25] E.D. Jiang, L. Wang, Z.P. Peng, Solving energy-efficient distributed job shop scheduling via multi-objective evolutionary algorithm with decomposition, *Swarm. Evol. Comput.* 58 (1) (2020) 100745, <https://doi.org/10.1016/j.swevo.2020.100745>.
- [26] H.C. Chang, T.K. Liu, Optimisation of distributed manufacturing flexible job shop scheduling by using hybrid genetic algorithms, *J. Intell. Manuf.* 28 (8) (2017) 1973–1986, <https://doi.org/10.1007/s10845-015-1084-y>.
- [27] L. Meng, C. Zhang, Y. Ren, B. Zhang, C. Lv, Mixed-integer linear programming and constraint programming formulations for solving distributed flexible job shop scheduling problem, *Comput. Ind. Eng.* 142 (1) (2020) 106347, <https://doi.org/10.1016/j.cie.2020.106347>.
- [28] R.H. Chen, B. Yang, S. Li, S.L. Wang, A self-learning genetic algorithm based on reinforcement learning for flexible job-shop scheduling problem, *Comput. Ind. Eng.* 149 (1) (2020) 106778, <https://doi.org/10.1016/j.cie.2020.106778>.
- [29] G. Gong, R. Chiong, Q. Deng, Q. Luo, A memetic algorithm for multi-objective distributed production scheduling: minimizing the makespan and total energy consumption, *J. Intell. Manuf.* 31 (2020 a) 1443–1466, <https://doi.org/10.1007/s10845-019-01521-9>.
- [30] Q. Luo, Q. Deng, G. Gong, X. Guo, X. Liu, A distributed flexible job shop scheduling problem considering worker arrangement using an improved memetic algorithm, *Expert. Syst. Appl.* 207 (2022 a) 117984, <https://doi.org/10.1016/j.eswa.2022.117984>.
- [31] J. Wang, H. Han, L. Wang, A feedback learning-based memetic algorithm for energy-aware distributed flexible job-shop scheduling with transportation constraints, *IEEE T Evol. Comput.* (2024), <https://doi.org/10.1109/TEVC.2024.3388527>.
- [32] Y. Zheng, N. Peng, H. Qi, G. Gong, D. Huang, K. Zhu, J. Liu, G. Liu, An improved memetic algorithm for distributed hybrid flow shop scheduling problem with operation inspection and reprocessing, *MEAS CONTROL-UK*, 00202940241245241, 2024, <https://doi.org/10.1177/00202940241245241>.
- [33] P.H. Lu, M.C. Wu, H. Tan, Y.H. Peng, C.F. Chen, A genetic algorithm embedded with a concise chromosome representation for distributed and flexible job-shop scheduling problems, *J. Intell. Manuf.* 29 (1) (2018) 19–34, <https://doi.org/10.1007/s10845-015-1083-z>.
- [34] W.X. Xu, Y.W. Hu, W. Luo, L. Wang, R. Wu, A multi-objective scheduling method for distributed and flexible job shop based on hybrid genetic algorithm and tabu search considering operation outsourcing and carbon emission, *Comput. Ind. Eng.* 157 (1) (2021) 107318, <https://doi.org/10.1016/j.cie.2021.107318>.
- [35] H. Li, X. Wang, J.J.E.S.W.A. Peng, A hybrid differential evolution algorithm for flexible job shop scheduling with outsourcing operations and job priority constraints, *Expert. Syst. Appl.* 201 (2022) 117182, <https://doi.org/10.1016/j.eswa.2022.117182>.
- [36] R. Li, W. Gong, L. Wang, C. Lu, Z. Pan, X. Zhuang, Double DQN-based coevolution for green distributed heterogeneous hybrid flowshop scheduling with multiple priorities of jobs, *IEEE T Autom. Sci. Eng.* 21 (4) (2023) 6550–6562, <https://doi.org/10.1109/TASE.2023.3327792>.
- [37] G. Gong, J. Tang, D. Huang, Q. Luo, K. Zhu, N. Peng, Energy-efficient flexible job shop scheduling problem considering discrete operation sequence flexibility, *Swarm. Evol. Comput.* 84 (2024) 101421, <https://doi.org/10.1016/j.swevo.2023.101421>.
- [38] H. Ma, Y. Zhang, S. Sun, T. Liu, Y. Shan, A comprehensive survey on NSGA-II for multi-objective optimization and applications, *Artif. Intell. Rev.* 56 (12) (2023) 15217–15270, <https://doi.org/10.1007/s10462-023-10526-z>.
- [39] F. Luan, H. Zhao, S.Q. Liu, Y. He, B. Tang, Enhanced NSGA-II for multi-objective energy-saving flexible job shop scheduling, *Sustain. Comput. Inform.* 39 (2023) 100901, <https://doi.org/10.1016/j.suscom.2023.100901>.
- [40] G. Gong, R. Chiong, Q. Deng, Q. Luo, A memetic algorithm for multi-objective distributed production scheduling: minimizing the makespan and total energy consumption, *Int. J. Prod. Res.* 31 (2020 b) 1443–1466, <https://doi.org/10.1007/s10845-019-01521-9>.
- [41] G.L. Gong, Q.W. Deng, R. Chiong, X.R. Gong, H.Z.Y. Huang, An effective memetic algorithm for multi-objective job-shop scheduling, *Knowl. Based. Syst.* 182 (1) (2019) 104840, <https://doi.org/10.1016/j.knsys.2019.07.011>.
- [42] Y. Xu, M. Zhang, M. Yang, D. Wang, Hybrid quantum particle swarm optimization and variable neighborhood search for flexible job-shop scheduling problem, *J. Manuf. Syst.* 73 (2024) 334–348, <https://doi.org/10.1016/j.jmsy.2024.02.007>.
- [43] R. Li, W. Gong, L. Wang, C. Lu, C. Dong, Co-evolution with deep reinforcement learning for energy-aware distributed heterogeneous flexible job shop scheduling, *IEEE T Syst. Man Cy S* 54 (1) (2023) 201–211, <https://doi.org/10.1109/TSMC.2023.3305541>.
- [44] W. Shao, Z. Shao, D. Pi, Lot sizing and scheduling problem in distributed heterogeneous hybrid flow shop and learning-driven iterated local search algorithm, *IEEE T Autom. Sci. Eng.* 21 (4) (2023) 6483–6497, <https://doi.org/10.1109/TASE.2023.3326301>.
- [45] Q. Luo, Q. Deng, G. Gong, X. Guo, X. Liu, A distributed flexible job shop scheduling problem considering worker arrangement using an improved memetic algorithm, *Expert. Syst. Appl.* (2022 b) 117984, <https://doi.org/10.1016/j.eswa.2022.117984>.
- [46] H. Zhang, C. Qin, W. Zhang, Z. Xu, G. Xu, Z. Gao, Energy-saving scheduling for flexible job shop problem with AGV transportation considering emergencies, *Systems* 11 (2) (2023) 103, <https://doi.org/10.3390/systems11020103>.
- [47] N. Peng, Y. Zheng, Z. Xiao, G. Gong, D. Huang, X. Liu, K. Zhu, Q. Luo, Multi-objective dynamic distributed flexible job shop scheduling problem considering uncertain processing time, *Cluster. Comput.* 28 (3) (2025) 185, <https://doi.org/10.1007/s10586-024-04803-x>.
- [48] G. Xu, Q. Bao, H. Zhang, Multi-objective green scheduling of integrated flexible job shop and automated guided vehicles, *Eng. Appl. Artif. Intell.* 126 (2023) 106864, <https://doi.org/10.1016/j.engappai.2023.106864>.
- [49] D. Chen, J. Guo, Integrated flexible job shop scheduling and machine layout problem considering mixed transportation ways, in: 2023 3rd International Conference on Computer Science, Electronic Information Engineering and Intelligent Control Technology (CEI), IEEE, 2023, pp. 737–741, <https://doi.org/10.1109/CEI60616.2023.10528015>.
- [50] Y. Yuan, H. Xu, Multiobjective flexible job shop scheduling using memetic algorithms, *IEEE T Autom. Sci. Eng.* 12 (1) (2013) 336–353, <https://doi.org/10.1109/TASE.2013.2274517>.
- [51] C.A.C. Coelho, N.C. Cortés, Solving multiobjective optimization problems using an artificial immune system, *Genet. Program. Evol. M* 6 (2005) 163–190, <https://doi.org/10.1016/j.procs.2018.08.114>.
- [52] Y. Fu, K. Gao, L. Wang, M. Huang, Y.-C. Liang, H. Dong, Scheduling stochastic distributed flexible job shops using an multi-objective evolutionary algorithm with simulation evaluation, *Int. J. Prod. Res.* 63 (1) (2025) 86–103, <https://doi.org/10.1080/00207543.2024.2356628>.
- [53] J. Ma, Y. Lei, Z. Wang, L. Jiao, R. Liu, A memetic algorithm based on immune multi-objective optimization for flexible job-shop scheduling problems, in: 2014 IEEE Congress on evolutionary computation (CEC), 2014, pp. 58–65, <https://doi.org/10.1109/CEC.2014.6900331>.
- [54] K. Deb, A. Pratap, S. Agarwal, T. Meyarivan, A fast and elitist multiobjective genetic algorithm: NSGA-II, *IEEE T Evol. Comput.* 6 (2) (2002) 182–197, <https://doi.org/10.1109/4235.996017>.
- [55] A. Asghari, M.K. Sohrabi, Bi-objective cloud resource management for dependent tasks using Q-learning and NSGA-3, *J. Amb. Intel. Hum. Comp.* 15 (1) (2024) 197–217, <https://doi.org/10.1007/s12652-022-03885-y>.



- [56] K. Zervoudakis, S. Tsafarakis, A mayfly optimization algorithm, *Comput. Ind. Eng.* 145 (2020) 106559, <https://doi.org/10.1016/j.cie.2020.106559>.
- [57] R. Salgotra, U. Singh, The naked mole-rat algorithm, *Neural Comput. Appl.* 31 (2019) 8837–8857, <https://doi.org/10.1007/s00521-019-04464-7>.
- [58] K. Kaur, U. Singh, R. Salgotra, An enhanced moth flame optimization, *Neural Comput. Appl.* 32 (2020) 2315–2349, <https://doi.org/10.1007/s00521-018-3821-6>.
- [59] K. Zervoudakis, S. Tsafarakis, A global optimizer inspired from the survival strategies of flying foxes, *Eng. Comput.-Germany* (2023) 1–34, <https://doi.org/10.1007/s00366-021-01554-w>.
- [60] B. Khurshid, S. Maqsood, A hybrid evolution strategies-simulated annealing algorithm for job shop scheduling problems, *Eng. Appl. Artif. Intell.* 133 (2024) 108016, <https://doi.org/10.1016/j.engappai.2024.108016>.
- [61] M. Abdel-Basset, R. Mohamed, D. El-Shahat, K.M. Sallam, An efficient hybrid optimization method for fuzzy flexible job-shop scheduling problem: steady-state performance and analysis, *Eng. Appl. Artif. Intell.* 123 (2023) 106249, <https://doi.org/10.1016/j.engappai.2023.106249>.